

Laboratory Record

submitted as part of

Programming for Data Science Lab (MACSE502)

LAB ASSESESMENT-4

by



Reg No: 26MML0031
Name: Hasnain H. Makada

To



Dr M Rajasekhara Babu

School of Computer Science and Engineering



JULY 2026

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
INDEX

Week No	Date	Sl. No.	Type	Topic	Title with download Link for document	Problem Statement	Page No.
1	09-07-2026	1	PP	Pandas	Basic Data Manipulation with Pandas	Create a DataFrame with columns Name, Age, and City containing data for five individuals. Perform the following operations: select only the Name and Age columns, filter rows where Age is greater than 25, add a new column Country with a default value, and sort the DataFrame by Age in descending order. Finally, calculate the average age of the individuals.	1
1	09-07-2026	2	EP	Pandas	AI Model Performance Analysis Using Pandas	An AI company maintains model performance records containing: • ModelID • Accuracy • Algorithm Perform: 1. Select ModelID and Accuracy. 2. Filter models with Accuracy greater than 90%. 3. Add Status with default value "Production Ready". 4. Sort models by Accuracy. 5. Calculate average model accuracy.	6
2	16-07-2026	3	PP	Pandas	Data Cleaning and Preprocessing with Pandas	Create a DataFrame with some missing values in columns Name, Age, and City. Perform the following operations: fill missing values in the Age column with the mean age, drop rows where Name or City is missing, and convert the Age column to integer type. Finally, normalize the Age column using min-max scaling	14
2	16-07-2026	4	EP	Pandas	Movie Rating Data Preprocessing Using Pandas	A streaming platform maintains movie records containing: • MovieID • Rating • Genre Perform: 1. Create a dataset with missing values. 2. Fill missing Rating values using average rating. 3. Remove rows with missing MovieID or Genre. 4. Convert Rating into integer type. Normalize rating values.	19

3	23-07-2026	5	PP	NumPy	Creating and Manipulating Arrays Using NumPy	Create a 2D array of shape (3, 4) with random integers between 0 and 10. Perform the following operations: calculate the mean and standard deviation of the entire array, slice the array to get the first two rows and last two columns, reshape the array to shape (4, 3), and perform element-wise multiplication with another array of the same shape.	30
3	23-07-2026	6	EP	NumPy	Warehouse Shipment and Logistic Analytics Using Numpy	A logistics company tracks shipments from 4 warehouses across 7 weeks. Create a 4 x 7 shipment matrix. Perform the following operations: (1) compute average shipments, (2) compute standard deviation, (3) slice the first two warehouses and last three weeks, (4) reshape the matrix into a 7 x 4 matrix, and (5) perform element-wise multiplication with a transportation-cost matrix of the same shape. As an extra	35
4	30/7/2026	7	PP	Pandas, Numpy, Matplotlib	Feature Engineering and Exploratory Data Analysis	Create a DataFrame with columns Feature1, Feature2, and Target containing random data. Perform the following operations: create a new feature that is the logarithm of Feature1, bin Feature2 into three categories (low, medium, high), and calculate the correlation matrix of the DataFrame. Finally, create a scatter plot of Feature1 vs. Feature2 colored by Target, and interpret any visible patterns.	46

4	30/7/2026	8	EP	Pandas, NumPy, Matplotlib	Employee Attrition Prediction Using Workforce Indicators	A company wants to analyze workforce behavior and predict employee attrition. Create a DataFrame containing: OvertimeHours JobSatisfactionScore SalaryLevel TrainingHours WorkLifeBalance YearsAtCompany PromotionCount AttritionStatus Perform: 1. Create Log_OvertimeHours. 2. Categorize JobSatisfactionScore into: - o Disengaged - o Satisfied - o Highly Engaged 3. Generate correlation matrix. 4. Create a Treemap Visualization of employee categories. Analyze workforce trends.	56
5	06/08/2026	9	PP	Web Scraping	requests, BeautifulSoup, pandas, json	Scrape data from a webpage and store it in a structured format like CSV or JSON	63

PP: Practice Program; EP: Exercise Program; CP: Challenging Program

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

Title	Data Manipulation and Analysis using the Starwars Dataset in R																					
Problem Statement	Basic Data Manipulation with Pandas: Create a DataFrame with columns Name, Age, and City containing data for five individuals. Perform the following operations: select only the Name and Age columns, filter rows where Age is greater than 25, add a new column Country with a default value, and sort the DataFrame by Age in descending order. Finally, calculate the average age of the individuals.																					
Objectives	- To understand how to create a Pandas DataFrame from a Python dictionary. - To learn column selection techniques using Pandas. - To apply filtering conditions on DataFrame rows. - To create and populate new columns in a DataFrame. - To sort data based on column values. - To perform basic statistical analysis using Pandas aggregate functions.																					
Dataset URL Link / File	<table><tr><td>Name</td><td>Age</td><td>City</td></tr><tr><td>Alice</td><td>23</td><td>New York</td></tr><tr><td>Bob</td><td>30</td><td>London</td></tr><tr><td>Charlie</td><td>28</td><td>Paris</td></tr><tr><td>David</td><td>35</td><td>Tokyo</td></tr><tr><td>Emma</td><td>22</td><td>Sydney</td></tr></table>				Name	Age	City	Alice	23	New York	Bob	30	London	Charlie	28	Paris	David	35	Tokyo	Emma	22	Sydney
Name	Age	City																				
Alice	23	New York																				
Bob	30	London																				
Charlie	28	Paris																				
David	35	Tokyo																				
Emma	22	Sydney																				
Functions Details	Sl. No.	Function	Package	Description of function																		
	1	DataFrame()	pandas	Creates a DataFrame																		
	2	sort_values()	pandas	Sorts DataFrame rows																		
	3	mean()	pandas	Calculates the average value																		
	4	sort_values()	pandas	Sorts DataFrame rows																		
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1 Import the pandas library and create a dictionary containing Name, Age and City Data Step 2 Convert the dictionary into a pandas DataFrame Step 3 Select only the Name and Age columns from the DataFrame. Step 4 Filter the DataFrame to display individuals whose age is greater than 25.																					

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

	Step 5 Add a new column named Country and assign a default value of India. Step 6 Sort the DataFrame in descending order based on the Age column. Step 7 Calculate the average age of all individuals using the mean() function. Step 8 – Display all intermediate and final results.
Program	<pre># Load Necessary Packages import pandas as pd # Step 1: Create Dataset dataframe1 = { "Name": ["Alice", "Bob", "Charlie", "David", "Emma"], "Age": [23, 30, 28, 35, 22], "City": ["New York", "London", "Paris", "Tokyo", "Sydney"] } # Step 2: Create DataFrame df = pd.DataFrame(dataframe1) print("Original DataFrame") print(df) print() # Step 3: Select Name and Age Columns name_age = df[["Name", "Age"]] print("Name and Age") print(name_age) print() # Step 4: Filter Rows where Age > 25 filtered_df = df[df["Age"] > 25] print("Age > 25") print(filtered_df) print()</pre>

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)

Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

```
# Step 5: Add Country
Column df["Country"] =
"India"

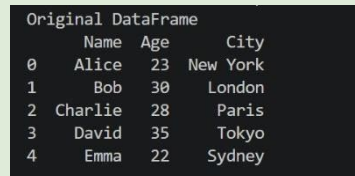
print("Country
Added") print(df)
print()

# Step 6: Sort by Age in Descending Order
sorted_df = df.sort_values(by="Age", ascending=False)

print("Sorted
Data")
print(sorted_df)
print()

# Step 7: Calculate Mean Age
mean_df = df["Age"].mean()

print("Mean Age")
print(mean_df)
```



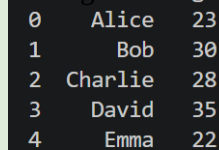
	Name	Age	City
0	Alice	23	New York
1	Bob	30	London
2	Charlie	28	Paris
3	David	35	Tokyo
4	Emma	22	Sydney

Figure 1.1 Original Dataframe

Figure 1.1 displays the original DataFrame created from a Python dictionary containing information about five individuals. The DataFrame consists of three columns: Name, Age, and City, where each row represents a different individual and their corresponding details. The pd.DataFrame() function converts

the dictionary data into a structured tabular format, making it easier to store, view, and manipulate data using Pandas. This DataFrame serves as the starting dataset for all subsequent operations such as Name and Age selection, filtering, adding new columns, sorting, and calculating Name and Age measures.

Figure 1.2 shows the DataFrame after selecting only the Name and Age columns from the original dataset using Pandas column indexing. This operation extracts the required columns and creates a new DataFrame named name_age, removing the City column from the view. Such column selection helps focus on relevant attributes for analysis and reduces unnecessary data processing.



	Name	Age
0	Alice	23
1	Bob	30
2	Charlie	28
3	David	35
4	Emma	22

Figure 1.2 Selected Name and Age Columns

**Output
Snapshots
With
Explanation/
Observations**

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

	Name	Age	City
1	Bob	30	London
2	Charlie	28	Paris
3	David	35	Tokyo

Figure 1.3: Filtered Data (Age > 25)

Figure 1.3 presents the filtered DataFrame containing only those individuals whose Age is greater than 25. This filtering operation is performed using a conditional expression in Pandas, where the Age column is checked against the condition `Age > 25`. As a result, only the rows

satisfying the condition are retained in the new DataFrame `filtered_df`. Figure 1.3 operation demonstrates how Pandas can be used to extract specific records based on user-defined criteria, which is a common requirement in data analysis and data preprocessing tasks.

	Name	Age	City	Country
0	Alice	23	New York	India
1	Bob	30	London	India
2	Charlie	28	Paris	India
3	David	35	Tokyo	India
4	Emma	22	Sydney	India

Figure 1.4: DataFrame with Country Column

Figure 1.4 displays the DataFrame after adding a new column named `Country` with a default value of "India" for all records. This operation is performed using Pandas column assignment, where the same value is assigned to every row in the newly created column. Adding new columns is a common data transformation technique used to enrich datasets with additional information. Figure 1.4 output confirms that the `Country` column has been successfully added to the DataFrame, and each individual has been assigned the default value India. The enriched dataset is now ready for further operations such as sorting and statistical analysis.

	Name	Age	City	Country
3	David	35	Tokyo	India
1	Bob	30	London	India
2	Charlie	28	Paris	India
0	Alice	23	New York	India
4	Emma	22	Sydney	India

Figure 1.5 Sorted Data by Age (Descending Order)

Figure 1.5 illustrates the DataFrame after sorting the records based on the Age column in descending order using the Pandas `sort_values()` function. In this operation, individuals are arranged from the highest age to the lowest age. Sorting helps organize data systematically and makes it easier to identify records with

the largest or smallest values. Figure 1.5 output confirms that the DataFrame has been successfully sorted in descending order of Age, placing the oldest individual at the top and the youngest individual at the bottom. Such sorting operations are commonly used in data analysis records efficiently.

Figure 1.6 Average Age of Individuals

Figure 1.6 presents the average age of all individuals in the dataset, calculated using the Pandas `mean()` function. The average (mean) age is obtained by summing all the age values and dividing the total by the number of individuals in the DataFrame. This operation demonstrates the use of aggregate statistical functions in Pandas for summarizing numerical data. Figure 1.6 output confirms that the Pandas `mean()` function has successfully computed the average age of the five individuals as 27.6 years. Such aggregate functions are widely used in data analysis to summarize and interpret numerical information efficiently.

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

Conclusions	This experiment successfully demonstrated the use of Pandas for basic data manipulation and analysis. A DataFrame containing the details of five individuals was created using the columns Name, Age, and City. Various DataFrame operations were then performed, including selecting specific columns, filtering rows based on a condition, adding a new column with default values, sorting records, and calculating statistical measures. These operations showcased the flexibility and efficiency of Pandas in handling tabular data. The results showed that: • The Name and Age columns could be extracted independently for focused analysis. • Filtering enabled the identification of individuals whose Age was greater than 25 years. • A new Country column was successfully added to enrich the dataset. • Sorting arranged the records in descending order of age, making it easy to identify the oldest and youngest individuals. • The average age of the individuals was calculated as 27.6 years, demonstrating the use of aggregate functions in data analysis. Overall, this exercise provided a strong foundation in DataFrame creation, data selection, filtering, transformation, sorting, and statistical analysis using Pandas. These fundamental data manipulation techniques are widely used in Data Science, Machine Learning, Data Analytics, and Business Intelligence applications, making them essential skills for working with real-world datasets.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018 2. Pandas Documentation: https://pandas.pydata.org/docs/ 3. Python Documentation: https://docs.python.org/3/ 4. Python Program (Google Colab link) https://colab.research.google.com/drive/1sjdIVacPTNg2JO_nGS8gic99hGEtMuRe?usp=sharing 5. Class lecture link https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

Title	Data Manipulation and Analysis using the Starwars Dataset in R				
Problem Statement	An AI company maintains model performance records containing: • ModelID • Accuracy • Algorithm Perform: 1. Select ModelID and Accuracy. 2. Filter models with Accuracy greater than 90%. 3. Add Status with default value "Production Ready". 4. Sort models by Accuracy. 5. Calculate average model accuracy.				
Objectives	• Understand how AI model performance data is stored and represented in a Pandas DataFrame. • Filter machine learning models based on conditions, such as accuracy greater than 90%. • Learn how to select specific columns such as ModelID and Accuracy. • Sort AI models according to their accuracy values. • Calculate statistical values such as average, minimum, and maximum model accuracy. • Rank machine learning models based on their performance.				
Dataset URL Link / File	ModelID	Algorithm	PreviousAccuracy	Accuracy	
	M101	Logistic Regression	84.5	87.2	
	M102	Decision Tree	87.0	89.5	
	M103	Random Forest	90.2	93.8	
	M104	Support Vector Machine	88.5	91.4	
	M105	K-Nearest Neighbors	82.5	85.6	
	M106	Random Forest	91.0	94.5	
	M107	Decision Tree	85.5	88.3	
	M108	Neural Network	92.0	96.1	
	M109	Logistic Regression	86.0	90.8	
	M110	Neural Network	93.1	95.4	
	M111	Gradient Boosting	89.4	92.7	
	M112	Gradient Boosting	91.8	94.1	
Functions Details	Sl. No.	Function	Package	Description of function	
	1	pd.DataFrame()	Pandas	Creates a DataFrame from the AI model data dictionary.	
	2	pd.read_csv()	Pandas	Reads AI model records from a CSV file.	
	3	dropna()	Pandas	Removes rows containing missing or invalid values	
	4	drop_duplicates()	Pandas	Removes duplicate records based on ModelID.	

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

	5	sort_values()	Pandas	Sorts the models according to their accuracy values.
	6	mean()	Pandas	Calculates the average accuracy of all AI models.
	7	groupby() and agg()	Pandas	Groups models by algorithm and calculates performance statistics.
	8	to_csv()	Pandas	Saves the analysed model results into a CSV file.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import the Pandas package. 2. Create the AI model dataset containing ModelID, Algorithm, PreviousAccuracy and Accuracy. 3. Convert the dataset into a Pandas DataFrame. 4. Clean the dataset by removing missing values, duplicate ModelIDs and invalid accuracy values. 5. Select only the ModelID and Accuracy columns. 6. Filter the models whose Accuracy is greater than 90%. 7. Add a Status column with the default value "Production Ready". 8. Change the status to "Needs Improvement" for models having Accuracy below 90%. 9. Sort the models in descending order based on Accuracy. 10. Assign ranks to the models, where the highest-accuracy model receives Rank 1. 11. Calculate the improvement of each model using: $\text{Improvement} = \text{Accuracy} - \text{PreviousAccuracy}$ 12. Calculate the average accuracy of all models. 13. Group the models by Algorithm and calculate minimum, maximum and average accuracy. 14. Identify the best-performing, lowest-performing and most-improved models. 15. Display all the analysis results in the console. 16. Generate an automated model evaluation report. 17. Save the analysed results in a CSV file and the report in a text file.			
Program	<pre>import sys from pathlib import Path import pandas as pd from analysis_core import (analyse_models, clean_data, create_algorithm_summary, create_report, create_sample_data, perform_basic_operations)</pre>			

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
#
=====
=====
# HEADING FUNCTION
# This keeps console output neat and
readable. #
=====
=====
def print_heading(title):
    print("\n" + "=" * 72)
    print(title)
    print("=" * 72)

#
=====
=====
# MAIN CONSOLE
PROGRAM #
# Run with sample data:
# python console_app.py
#
# Run with CSV data:
# python console_app.py
sample_ai_models.csv #
=====
=====
def main():
    print_heading("AI MODEL PERFORMANCE ANALYSIS
- CONSOLE MODE")

    csv_file = None

    # If the user gives a file path, use that
    CSV. if len(sys.argv) > 1:
        csv_file = sys.argv[1]

    if csv_file:
        csv_path = Path(csv_file)

        if not csv_path.exists():
            print(f"Error: File not found:
{csv_path}") return

        print(f"Using CSV file: {csv_path}")
        raw_data = pd.read_csv(csv_path)

    else:
        print("Using built-in sample data.")
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
raw_data = create_sample_data()

try:
    cleaned_data =
clean_data(raw_data) except
ValueError as error:
    print(f'Data error: {error}')
    return

analysed_data =
    analyse_models( cleaned_data
    , production_threshold=90.0
)

results =
    perform_basic_operations( analy
    sed_data
)

print_heading("ORIGINAL CLEANED DATA")
print(
    cleaned_data.to_string(index=False)
)

print_heading("1. SELECT MODELID AND
ACCURACY") print(
    results["selected_columns"]
    .to_string(index=False)
)

print_heading(
    "2. FILTER MODELS WITH ACCURACY GREATER
    THAN 90%"
)
print(
    results["models_above_90"][[
        ["ModelID", "Algorithm",
        "Accuracy"]
    ].to_string(index=False)
)

print_heading(
    '3. ADD STATUS WITH DEFAULT "PRODUCTION READY"'
)
print(
    results["status_table"]
    .to_string(index=False)
)
print_heading("4. SORT MODELS BY ACCURACY") print(
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
results["sorted_models"][\n    ["ModelID", "Algorithm", "Accuracy"]\n].to_string(index=False)\n)\n\nprint_heading("5. CALCULATE AVERAGE MODEL\nACCURACY")\nprint(\n    f'Average model accuracy: '\n    f'{results["average_accuracy"]:.2f}%\n')\n\nprint_heading("MODEL RANKING")\nprint(\n    analysed_data[\n        [\n            "Rank",\n            "ModelID",\n            "Algorithm",\n            "PreviousAccuracy",\n            "Accuracy",\n            "Improvement",\n            "Status"\n        ]\n    ].to_string(index=False)\n)\n\nprint_heading("ALGORITHM PERFORMANCE COMPARISON")\nalgorithm_summary =\n    create_algorithm_summary( analysed_data\n)\n\nprint(\n    algorithm_summary.to_string(index=False)\n)\n\nprint_heading("ACCURACY IMPROVEMENT TRACKING")\nprint(\n    analysed_data[\n        [\n            "ModelID",\n            "PreviousAccuracy",\n            "Accuracy",\n            "Improvement",\n            "ImprovementStatus"\n        ]\n    ].to_string(index=False)\n)\n)
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
report = create_report(  
    analysed_data,  
    production_threshold=90.0  
)  
  
print_heading("AUTOMATED EVALUATION  
REPORT") print(report)  
  
# Save analysed  
data.  
analysed_data.to_csv(  
    "console_ai_model_results.csv",  
    index=False  
)  
  
# Save the report.  
with open(  
    "console_ai_model_report.txt",  
    "w",  
    encoding="utf-8"  
) as report_file:  
    report_file.write(report)  
  
print_heading("FILES CREATED")  
print("console_ai_model_results.csv")  
print("console_ai_model_report.txt")  
  
if __name__ == "__main__":  
    main()
```

**Output
Snapshots
With
Explanation/
Observations**

ORIGINAL CLEANED DATA

ModelID	Algorithm	PreviousAccuracy	Accuracy
M101	Logistic Regression	84.5	87.2
M102	Decision Tree	87.0	89.5
M103	Random Forest	90.2	93.8
M104	Support Vector Machine	88.5	91.4
M105	K-Nearest Neighbors	82.5	85.6
M106	Random Forest	91.0	94.5
M107	Decision Tree	85.5	88.3
M108	Neural Network	92.0	96.1
M109	Logistic Regression	86.0	90.8
M110	Neural Network	93.1	95.4
M111	Gradient Boosting	89.4	92.7
M112	Gradient Boosting	91.8	94.1

Figure 2.1 Original Cleaned Dataset

The ModelID and Accuracy columns were selected successfully. Models having accuracy greater than 90% were filtered from the dataset.

The program successfully loaded and cleaned the AI model performance dataset. The dataset contained ModelID, Algorithm, PreviousAccuracy and Accuracy.

1. SELECT MODELID AND ACCURACY

ModelID	Accuracy
M108	96.1
M110	95.4
M106	94.5
M112	94.1
M103	93.8
M111	92.7
M104	91.4
M109	90.8
M102	89.5
M107	88.3
M101	87.2
M105	85.6

Figure 1.2 Selected ModelID and Accuracy

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

2. FILTER MODELS WITH ACCURACY GREATER THAN 90%

ModelID	Algorithm	Accuracy
M108	Neural Network	96.1
M110	Neural Network	95.4
M106	Random Forest	94.5
M112	Gradient Boosting	94.1
M103	Random Forest	93.8
M111	Gradient Boosting	92.7
M104	Support Vector Machine	91.4
M109	Logistic Regression	90.8

Figure 1.3 Models Accuracy Greater than 90%

A Status column was added. Models with accuracy greater than or equal to 90% were marked as "Production Ready", while the remaining

3. ADD STATUS WITH DEFAULT "PRODUCTION READY"

ModelID	Accuracy	Status
M108	96.1	Production Ready
M110	95.4	Production Ready
M106	94.5	Production Ready
M112	94.1	Production Ready
M103	93.8	Production Ready
M111	92.7	Production Ready
M104	91.4	Production Ready
M109	90.8	Production Ready
M102	89.5	Needs Improvement
M107	88.3	Needs Improvement
M101	87.2	Needs Improvement
M105	85.6	Needs Improvement

Figure 1.4 Status Column and Models sorted by accuracy

5. CALCULATE AVERAGE MODEL ACCURACY
Average model accuracy: 91.62%
Improvement

MODEL RANKING

Rank	ModelID	Algorithm	PreviousAccuracy	Accuracy	Improvement	Status
1	M108	Neural Network	92.0	96.1	4.1	Production Ready
2	M110	Neural Network	93.1	95.4	2.3	Production Ready
3	M106	Random Forest	91.0	94.5	3.5	Production Ready
4	M112	Gradient Boosting	91.8	94.1	2.3	Production Ready
5	M103	Random Forest	90.2	93.8	3.6	Production Ready
6	M111	Gradient Boosting	89.4	92.7	3.3	Production Ready
7	M104	Support Vector Machine	88.5	91.4	2.9	Production Ready
8	M109	Logistic Regression	86.0	90.8	4.8	Production Ready
9	M102	Decision Tree	87.0	89.5	2.5	Needs Improvement
10	M107	Decision Tree	85.5	88.3	2.8	Needs Improvement
11	M101	Logistic Regression	84.5	87.2	2.7	Needs Improvement
12	M105	K-Nearest Neighbors	82.5	85.6	3.1	Needs Improvement

Figure 1.5 Average Model Accuracy

The average accuracy of all models was calculated. Accuracy improvement was also calculated by subtracting PreviousAccuracy from the current Accuracy.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

ALGORITHM PERFORMANCE COMPARISON						
Algorithm	ModelCount	MinimumAccuracy	MaximumAccuracy	AverageAccuracy	AverageImprovement	
Neural Network	2	95.4	96.1	95.75	3.20	
Random Forest	2	93.8	94.5	94.15	3.55	
Gradient Boosting	2	92.7	94.1	93.40	2.80	
Support Vector Machine	1	91.4	91.4	91.40	2.90	
Logistic Regression	2	87.2	90.8	89.00	3.75	
Decision Tree	2	88.3	89.5	88.90	2.65	
K-Nearest Neighbors	1	85.6	85.6	85.60	3.10	

Figure 1.6 Models Grouped according to their algorithm and other metrics

The models were grouped according to their algorithms, and minimum, maximum and average accuracy values were calculated for each algorithm.

```
SUMMARY
Total models analysed: 12
Production threshold: 90.0%
Average model accuracy: 91.62%
Production-ready models: 8
Models needing improvement: 4

BEST MODEL
Model ID: M108
Algorithm: Neural Network
Accuracy: 96.10%

LOWEST-PERFORMING MODEL
Model ID: M105
Algorithm: K-Nearest Neighbors
Accuracy: 85.60%

MOST IMPROVED MODEL
Model ID: M109
Algorithm: Logistic Regression
Improvement: 4.80 percentage points

BEST ALGORITHM
Algorithm: Neural Network
Average accuracy: 95.75%
```

The program identified the best-performing model, lowest-performing model, most-improved model and best-performing algorithm.

Figure 1.7 Final Output

Conclusions

The exercise successfully demonstrated how Pandas can be used to analyse AI model performance data. The program selected, filtered, sorted and ranked the models, calculated average accuracy, compared algorithm performance and tracked accuracy improvement. It also generated an automated evaluation report and saved the final results in CSV and text formats.

References

1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018
2. Python Program
https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%201/Excercise_Program_Console.py
3. Class lecture link
https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view?usp=drive_link

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

Title	Student Marks Data Manipulation and Analysis using Pandas																																	
Problem Statement	Data Cleaning and Preprocessing with Pandas: Create a DataFrame with some missing values in columns Name, Age, and City. Perform the following operations: fill missing values in the Age column with the mean age, drop rows where Name or City is missing, and convert the Age column to integer type. Finally, normalize the Age column using min-max scaling.																																	
Objectives	● To understand how to create a Pandas DataFrame containing missing values. ● To learn how to identify and handle missing data using the fillna() function. ● To remove incomplete records using the dropna() function. ● To convert a DataFrame column to the required data type using the astype() function. ● To normalize numerical data using the Min-Max Scaling technique. To understand the importance of data preprocessing before performing data analysis and machine learning tasks.																																	
Dataset URL Link / File	<table><tr><th>Name</th><th>Age</th><th>City</th></tr><tr><td>Arjun</td><td>np.nan</td><td>np.nan</td></tr><tr><td>Priya</td><td>28</td><td>Mumbai</td></tr><tr><td>Rohan</td><td>35</td><td>Delhi</td></tr><tr><td>Sneha</td><td>np.nan</td><td>np.nan</td></tr><tr><td>Amit</td><td>40</td><td>Bangalore</td></tr><tr><td>Deepa</td><td>30</td><td>Chennai</td></tr><tr><td>Rahul</td><td>np.nan</td><td>np.nan</td></tr><tr><td>np.nan</td><td>33</td><td>Kolkata</td></tr><tr><td>np.nan</td><td>29</td><td>Ahmedabad</td></tr></table>				Name	Age	City	Arjun	np.nan	np.nan	Priya	28	Mumbai	Rohan	35	Delhi	Sneha	np.nan	np.nan	Amit	40	Bangalore	Deepa	30	Chennai	Rahul	np.nan	np.nan	np.nan	33	Kolkata	np.nan	29	Ahmedabad
Name	Age	City																																
Arjun	np.nan	np.nan																																
Priya	28	Mumbai																																
Rohan	35	Delhi																																
Sneha	np.nan	np.nan																																
Amit	40	Bangalore																																
Deepa	30	Chennai																																
Rahul	np.nan	np.nan																																
np.nan	33	Kolkata																																
np.nan	29	Ahmedabad																																
Functions Details	Sl. No.	Function	Package	Description of function																														
	1	DataFrame()	pandas	Creates a DataFrame from structured data																														
	2	Mean()	pandas	Calculates the average value																														
	3	fillna()	pandas	Replaces missing values with a specified value.																														
	4	dropna()	pandas	Removes rows containing missing values.																														

Student Reg. No: 26MML0031

Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1 Import the pandas library. Step 2 Create a Python dictionary containing StudentID, Name, and Marks for five students. Step 3 Convert the dictionary into a Pandas DataFrame and display the original dataset. Step 4 Select and display only the Name and Marks columns. Step 5 Filter and display students whose Marks are greater than 70. Step 6 Add a Grade column by assigning grades A to E according to the marks obtained. Step 7 Display the updated DataFrame containing the Grade column. Step 8 Sort the DataFrame by Marks in ascending order and display the result. Step 9 Calculate and display the average marks using the mean() function.
Program	<pre># Import the Pandas library import pandas as pd # Create a dictionary containing student details dictionary_1 = { "StudentID": [10, 11, 12, 13, 14], "Name": ["Hasnain", "John", "Alice", "Sahil", "Cook"], "Marks": [45, 86, 90, 67, 97] } # Convert the dictionary into a Pandas DataFrame dataframe = pd.DataFrame(dictionary_1) # Display the original dataset print("Diplaying the Original Dataset\ n") print(dataFrame, "\n") # Select and display only the Name and Marks columns only_name_colum = pd.DataFrame(dataframe, columns=["Name", "Marks"]) print("Displaying only name and marks columns\n") print(only_name_colum, "\n") # Filter students whose marks are greater than 70 marks_greater_than_70 = pd.DataFrame(dataframe[dataFrame["Marks"] > 70]) print("Displaying students with marks greater than 70\n") print(marks_greater_than_70, "\n") # Assign grades based on the marks obtained dataFrame["Grade"] = ["A" if marks >= 90 else "B" if marks >= 80 else "C" if marks >= 70 else "D" if marks >= 60 else "E" for marks in dataFrame["Marks"]]</pre>

Student Reg. No: 26MML0031

Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

```
]

# Display the updated dataset with the Grade column
print("Displaying the updated dataset\n")
print(dataFrame, "\n")

# Sort the dataset by Marks in ascending order
sort_by_marks = dataFrame.sort_values(
    by="Marks", ascending=True
)
print("Displaying the sorted dataset\n")
print(sort_by_marks, "\n")

# Calculate and display the average
marks average =
dataFrame["Marks"].mean()
print("Average Marks \n")
print(average)
```

Displaying the Original Dataset

```
***
   StudentID  Name  Marks
0         10  Hasnain    45
1         11   John    86
2         12   Alice    90
3         13   Sahil    67
4         14    Cook    97
```

Figure 2.1 Original Student Dataset

Figure 2.1 displays the original DataFrame created from student dictionary. It contains the columns StudentID, Name, and Marks for five students. The pd.DataFrame function converts the dictionary into a structured format that can be analysed using Pandas.

2.2 shows only the Name and Marks columns selected from the original DataFrame. This demonstrates column selection and allows the analysis to focus on the student names and their corresponding marks without displaying StudentID.

Displaying only name and marks columns

```
   Name  Marks
0  Hasnain    45
1    John    86
2   Alice    90
3   Sahil    67
4    Cook    97
```

Figure 2.2 Selected Name and Marks Columns

**Output
Snapshots
With
Explanation/
Observations**

Displaying students with marks greater than 70

```
   StudentID  Name  Marks
1         11   John    86
2         12   Alice    90
4         14    Cook    97
```

Figure 2.3 Filtering student with marks > 70

Figure 2.3 presents the filtered records for students whose marks are greater than 70. The Boolean condition `dataFrame["Marks"] > 70` retains John, Alice, and Cook because their marks satisfy the specified condition.

2.4 displays the updated DataFrame after adding the Grade column. Grades are assigned using conditional logic: A for marks of 90 or above, B for 80-89, C for 70-79, D for 60-69, and marks below 60.

Displaying the updated dataset

```
   StudentID  Name  Marks  Grade
0         10  Hasnain    45     E
1         11   John    86     B
2         12   Alice    90     A
3         13   Sahil    67     D
4         14    Cook    97     A
```

Figure 2.4 Dataset after Adding the Grade Column

Displaying the sorted dataset

```
   StudentID  Name  Marks  Grade
0         10  Hasnain    45     E
3         13   Sahil    67     D
1         11   John    86     B
2         12   Alice    90     A
4         14    Cook    97     A
```

2.5 Dataset Sorted by Marks in Ascending Order

Figure 2.5 shows the records sorted from the lowest to the highest marks using `sort_values()`. Hasnain appears first with 45 marks, while Cook appears last with 97 marks. The original row indexes are retained by Pandas.

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

Figure using five of	2.6 displays the average marks calculated the mean() function. The total marks of all students divided by five gives an average 77.0.	Average Marks 77.0 Figure 2.6 Average Marks
Conclusions	This exercise successfully demonstrated basic student-data manipulation and analysis using Pandas. A DataFrame containing StudentID, Name, and Marks was created from a Python dictionary, and multiple operations were performed on the dataset. The results showed that: • The Name and Marks columns were selected successfully for focused analysis. • Students with marks greater than 70 were identified using Boolean filtering. • A Grade column was generated using conditional logic based on each student's marks. • The records were arranged in ascending order of marks using sort_values(). • The average marks of the five students were calculated as 77.0 using mean(). Overall, the exercise provided practical experience in DataFrame creation, column selection, row filtering, conditional transformation, sorting, and statistical aggregation. These operations are fundamental for data preprocessing and exploratory data analysis in Data Science and Machine Learning applications.	
References	1. Wes McKinney, Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter, 3rd Edition, O'Reilly Media, 2022. 2. Pandas Documentation: https://pandas.pydata.org/docs/ 3. Python Documentation: https://docs.python.org/3/ 4. Python Program (Google Colab Link) https://colab.research.google.com/drive/1qBzvtH6ZEis8Jv20WDiYXKI4PXGSXcWJ?usp=sharing 4. Pandas DataFrame API Reference: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html 5. Class lecture link: https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view	

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

Title	Data Manipulation and Analysis using the Starwars Dataset in R					
Problem Statement	A streaming platform maintains movie records containing: • MovieID • Rating • Genre Perform: 1. Create a dataset with missing values. 2. Fill missing Rating values using average rating. 3. Remove rows with missing MovieID or Genre. 4. Convert Rating into integer type. 5. Normalize rating values.					
Objectives	• To understand movie data preprocessing using the Pandas library. • To create a dataset containing missing values. • To identify and handle missing values in a dataset. • To fill missing Rating values using the average rating. • To remove records with missing MovieID or Genre. • To convert the Rating column into integer data type. • To normalize movie rating values using Min-Max normalization. • To identify top-rated movies based on ratings.					
Dataset URL Link / File	MovieID	MovieTitle	Rating	Genre	ViewCount	Likes
	MV101	The Last Horizon	8.5	Science Fiction	125000	11200
	MV102	City Lights		Comedy	98000	7500
	MV103	Hidden Truth	7.8	Thriller	110000	8900
	None	Lost Kingdom	9.1	Fantasy	150000	13500
	MV105	Digital Dreams	8.9	Science Fiction	175000	16000
	MV106	Silent River		Drama	87000	6100
	MV107	Future Code	9.3	Science Fiction	210000	19800
	MV108	Broken Compass	6.7		65000	4200
	MV109	Midnight Journey	8.1	Adventure	132000	10400
	MV110	Laugh Again	7.5	Comedy	105000	8100
	MV111	Ocean Mystery	8.8	Mystery	143000	12100
	MV112	Final Mission		Action	190000	17400

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

	MV113	Parallel Lives	8.4	Drama	118000	9600
	MV114	Wild Trails	7.2	Adventure	99000	7200
	MV115	The Great Escape	9.0	Action	205000	18800
Functions Details	Sl. No.	Function	Package	Description of function		
	1	pd.DataFrame()	Pandas	Creates a DataFrame from the movie dataset stored in dictionary format.		
	2	isnull()	Pandas	Identifies missing values present in the movie dataset.		
	3	mean()	Pandas	Fills missing Rating values using the calculated average rating.		
	4	dropna()	Pandas	Removes rows containing missing MovieID or Genre values.		
	5	pd.to_numeric()	Pandas	Converts Rating values into numeric format and handles invalid entries.		
	6	round()	Pandas	Rounds decimal Rating values before converting them into integers.		
	7	astype()	Pandas	Converts the Rating column into integer data type.		
	8	min()	Pandas	Finds the minimum Rating value required for normalization.		
	9	max()	Pandas	Finds the maximum Rating value required for normalization.		
	10	groupby()	Pandas	Groups movie records according to Genre for genre-wise analysis.		
	11	sort_values()	Pandas	Sorts movies based on rating, popularity score or number of views.		
	12	fillna()	Pandas	Fills missing Rating values using the calculated average rating.		
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import the Pandas library and create the movie dataset in dictionary format.					
	Step 2: Convert the dictionary into a Pandas DataFrame using pd.DataFrame().					
	Step 3: Display the original dataset containing missing values.					
	Step 4: Calculate the average value of the Rating column using the mean() function.					
	Step 5: Fill the missing Rating values using fillna() with the calculated average rating.					

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

	Step 6: Remove rows containing missing MovieID or Genre using the dropna() function. Step 7: Convert the Rating column into numeric format using pd.to_numeric() and then convert it into integer type using round() and astype(int). Step 8: Normalize the Rating values using the Min-Max Normalization formula: Normalized Rating = (Rating – Minimum Rating) / (Maximum Rating – Minimum Rating) Step 9: Calculate the Engagement Rate and Popularity Score for each movie based on views, likes and ratings. Step 10: Sort the movies according to Popularity Score and Rating using sort_values() and assign popularity ranks. Step 11: Generate a genre-wise report using the groupby() function to compare movie performance across different genres. Step 12: Recommend the top movies based on selected genre and minimum rating, generate the automated analytics report, and save the processed results for future analysis.
Program	<pre>import pandas as pd # # 1. CREATE MOVIE DATASET IN DICTIONARY FORMAT # movie_data = { "MovieID": ["MV101", "MV102", "MV103", None, "MV105", "MV106", "MV107", "MV108", "MV109", "MV110", "MV111", "MV112", "MV113", "MV114", "MV115"], "MovieTitle": ["The Last Horizon", "City Lights", "Hidden Truth", "Lost Kingdom", "Digital Dreams", "Silent River", "Future Code", "Broken Compass", "Midnight Journey",] }</pre>

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
"Laugh Again",
"Ocean Mystery",
"Final Mission",
"Parallel Lives",
"Wild Trails",
"The Great
Escape"
],

"Rating": [
    8.5, None, 7.8, 9.1, 8.9,
    None, 9.3, 6.7, 8.1, 7.5,
    8.8, None, 8.4, 7.2, 9.0
],

"Genre": [
    "Science
    Fiction",
    "Comedy",
    "Thriller",
    "Fantasy",
    "Science
    Fiction",
    "Drama",
    "Science
    Fiction", None,
    "Adventure",
    "Comedy",
    "Mystery",
    "Action",
    "Drama",
    "Adventure",
    "Action"
],

"ViewCount": [
    125000, 98000, 110000, 150000, 175000,
    87000, 210000, 65000, 132000, 105000,
    143000, 190000, 118000, 99000, 205000
],

"Likes": [
    11200, 7500, 8900, 13500, 16000,
    6100, 19800, 4200, 10400, 8100,
    12100, 17400, 9600, 7200, 18800
]
} -----
# 2. CONVERT DICTIONARY INTO DATAFRAME
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
# -----
movies = pd.DataFrame(movie_data)

print("\nORIGINAL MOVIE DATASET")
print("=" * 90)
print(movies.to_string(index=False))

# -----
# 3. CALCULATE AVERAGE
RATING # -----
average_rating = movies["Rating"].mean()

print("\nAVERAGE RATING")
print("=" * 90)
print("Average Rating:", round(average_rating, 2))

# -----
# 4. FILL MISSING RATING
VALUES # -----
movies["Rating"] = movies["Rating"].fillna(average_rating)

print("\nDATASET AFTER FILLING MISSING RATING VALUES")
print("=" * 90)
print(
    movies[
        ["MovieID", "MovieTitle", "Rating", "Genre"]
    ].to_string(index=False)
)

# -----
# 5. REMOVE ROWS WITH MISSING MOVIEID OR
GENRE # -----
movies = movies.dropna(subset=["MovieID", "Genre"])

print("\nDATASET AFTER REMOVING MISSING MOVIEID OR
GENRE")
print("=" * 90)
print(
    movies[
        ["MovieID", "MovieTitle", "Rating", "Genre"]
    ].to_string(index=False)
)

# -----
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
# 6. CONVERT RATING INTO INTEGER
TYPE #-----
movies["Rating"] = movies["Rating"].round().astype(int)

print("\nRATING CONVERTED INTO INTEGER TYPE")
print("=" * 90)
print(
    movies[
        ["MovieID", "MovieTitle", "Rating"]
    ].to_string(index=False)
)

print("\nRating Data Type:", movies["Rating"].dtype)

# -----
# 7. NORMALIZE RATING VALUES
# Min-Max
Normalization #-----
minimum_rating = movies["Rating"].min()
maximum_rating = movies["Rating"].max()

movies["NormalizedRating"] = (
    (movies["Rating"] -
     minimum_rating)
    / (maximum_rating - minimum_rating)
).round(3)

print("\nNORMALIZED RATING VALUES")
print("=" * 90)
print(
    movies[
        [
            "MovieID",
            "MovieTitle",
            "Rating",
            "NormalizedRating"
        ]
    ].to_string(index=False)
)

# -----
# 8. CALCULATE ENGAGEMENT
RATE #-----
movies["EngagementRate"] = (
    movies["Likes"] / movies["ViewCount"] * 100
).round(2)
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
print("\nVIEWER ENGAGEMENT")
print("=" * 90)
print(
    movies[
        [
            "MovieTitle",
            "ViewCount",
            "Likes",
            "EngagementRate"
        ]
    ].to_string(index=False)
)

# -----
# 9. CALCULATE POPULARITY SCORE
# -----
maximum_engagement = movies["EngagementRate"].max()

movies["NormalizedEngagement"] = (
    movies["EngagementRate"] / maximum_engagement
).round(3)

movies["PopularityScore"] = (
    movies["NormalizedRating"] *
    0.60
    + movies["NormalizedEngagement"] * 0.40
).round(3)

# -----
# 10. SORT AND RANK
MOVIES # -----
movies = movies.sort_values(
    by=["PopularityScore", "Rating"],
    ascending=[False, False]
).reset_index(drop=True)

movies["PopularityRank"] = range(1, len(movies) +

1) print("\nTOP-RATED AND POPULAMOVIES")
print("=" * 90)
print(
    movies[
        [
            "PopularityRank",
            "MovieID",
            "MovieTitle",
            "Genre",
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
"Rating",
"ViewCount",
"PopularityScore"
]
].to_string(index=False)
)

# -----
# 11. GENERATE GENRE-WISE REPORT
# -----

genre_report =
    movies.groupby( "Genre",
                    as_index=False
                ).agg(
                    MovieCount=("MovieID", "count"),
                    AverageRating=("Rating", "mean"),
                    HighestRating=("Rating", "max"),
                    TotalViews=("ViewCount", "sum"),
                    TotalLikes=("Likes", "sum")
                )

genre_report["AverageRating"] = (
    genre_report["AverageRating"].round(2)
)

genre_report =
    genre_report.sort_values( by="AverageRating", ascending=False
                             )

print("\nGENRE-WISE MOVIE REPORT")
print("=" * 90)
print(genre_report.to_string(index=False))

# -----
# 12. SIMPLE MOVIE
# RECOMMENDATION # -----
preferred_genre = "Science Fiction"
minimum_required_rating = 7

recommended_movies = movies[
    (movies["Genre"] ==
     preferred_genre)
    & (movies["Rating"] >= minimum_required_rating)
].sort_values( by="PopularityScore", ascending=False
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
)

print("\nMOVIE RECOMMENDATIONS")
print("=" * 90)
print("Preferred Genre:", preferred_genre)
print("Minimum Rating:",
minimum_required_rating)

print(
    recommended_movies[
        [
            "MovieID",
            "MovieTitle",
            "Genre",
            "Rating",
            "PopularityScore"
        ]
    ].to_string(index=False)
)

# -----
# 13. AUTOMATED ANALYTICS REPORT
# -----
top_movie = movies.iloc[0]

most_viewed_movie =
    movies.loc[ movies["ViewCount"].idxmax()
]

best_genre = genre_report.iloc[0]

print("\nAUTOMATED MOVIE ANALYTICS REPORT")
print("=" * 90)

print("Total Movies Analysed:", len(movies))
print("Average Movie Rating:", round(movies["Rating"].mean(), 2))
print("Total Genres:", movies["Genre"].nunique())

print("\nTop-Rated and Most Popular Movie")
print("Movie Title:", top_movie["MovieTitle"])
print("Genre:", top_movie["Genre"])
print("Rating:", top_movie["Rating"])
print("Popularity Score:", top_movie["PopularityScore"])

print("\nMost Viewed Movie")
print("Movie Title:",
most_viewed_movie["MovieTitle"]) print("Views:",
most_viewed_movie["ViewCount"])
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
print("\nBest Performing Genre")
print("Genre:", best_genre["Genre"])
print("Average Rating:", best_genre["AverageRating"])

# -----
# 14. SAVE RESULTS
# -----

movies.to_csv( "processed_movie
_data.csv", index=False
)

genre_report.to_csv( "genre_wise
_report.csv", index=False
)

print("\nFILES CREATED")
print("=" * 90)
print("processed_movie_data.csv")
print("genre_wise_report.csv")
```

**Output
Snapshots
With
Explanation/
Observations**

1. ORIGINAL DATASET WITH MISSING VALUES

MovieID	MovieTitle	Rating	Genre	ViewCount	Likes
MV101	The Last Horizon	8.5	Science Fiction	125000	11200
MV102	City Lights	NaN	Comedy	98000	7500
MV103	Hidden Truth	7.8	Thriller	110000	8900
NaN	Lost Kingdom	9.1	Fantasy	150000	13500
MV105	Digital Dreams	8.9	Science Fiction	175000	16000
MV106	Silent River	NaN	Drama	87000	6100
MV107	Future Code	9.3	Science Fiction	210000	19800
MV108	Broken Compass	6.7	NaN	65000	4200
MV109	Midnight Journey	8.1	Adventure	132000	10400
MV110	Laugh Again	7.5	Comedy	105000	8100
MV111	Ocean Mystery	8.8	Mystery	143000	12100
MV112	Final Mission	NaN	Action	190000	17400
MV113	Parallel Lives	8.4	Drama	110000	9600
MV114	Wild Trails	7.2	Adventure	99000	7200
MV115	The Great Escape	9.0	Action	205000	18800

Figure 1.1 Original dataset printed after creating the DataFrame (containing missing values).

The average rating was calculated successfully and was used to replace all missing Rating values.

2. FILL MISSING RATING USING AVERAGE RATING

Average Rating used: 8.28

MovieID	MovieTitle	Rating
MV107	Future Code	9
MV115	The Great Escape	9
MV105	Digital Dreams	9
MV111	Ocean Mystery	9
MV112	Final Mission	8
MV101	The Last Horizon	8
MV113	Parallel Lives	8
MV103	Hidden Truth	8
MV109	Midnight Journey	8
MV110	Laugh Again	8
MV102	City Lights	8
MV106	Silent River	8
MV114	Wild Trails	7

Figure 1.2 Output showing the calculated average rating.

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

3. REMOVE ROWS WITH MISSING MOVIEID OR GENRE

MovieID	MovieTitle	Genre	Rating
MV107	Future Code	Science Fiction	9
MV115	The Great Escape	Action	9
MV105	Digital Dreams	Science Fiction	9
MV111	Ocean Mystery	Mystery	9
MV112	Final Mission	Action	8
MV101	The Last Horizon	Science Fiction	8
MV113	Parallel Lives	Drama	8
MV103	Hidden Truth	Thriller	8
MV109	Midnight Journey	Adventure	8
MV110	Laugh Again	Comedy	8
MV102	City Lights	Comedy	8
MV106	Silent River	Drama	8
MV114	Wild Trails	Adventure	7

Figure 1.3 Dataset after fillna() operation.

Missing Rating values were replaced with the calculated average rating while preserving all other records.

Rating values were converted into integer type and normalized using the Min-Max Normalization technique.

5. NORMALIZE RATING VALUES

MovieID	MovieTitle	Rating	NormalizedRating
MV107	Future Code	9	1.0
MV115	The Great Escape	9	1.0
MV105	Digital Dreams	9	1.0
MV111	Ocean Mystery	9	1.0
MV112	Final Mission	8	0.5
MV101	The Last Horizon	8	0.5
MV113	Parallel Lives	8	0.5
MV103	Hidden Truth	8	0.5
MV109	Midnight Journey	8	0.5
MV110	Laugh Again	8	0.5
MV102	City Lights	8	0.5
MV106	Silent River	8	0.5
MV114	Wild Trails	7	0.0

Figure 1.4 Integer Rating output along with the **NormalizedRating** column.

TOP-RATED AND POPULAR MOVIES

PopularityRank	MovieID	MovieTitle	Genre	Rating	ViewCount	PopularityScore
1	MV107	Future Code	Science Fiction	9	210000	1.000
2	MV115	The Great Escape	Action	9	205000	0.989
3	MV105	Digital Dreams	Science Fiction	9	175000	0.988
4	MV111	Ocean Mystery	Mystery	9	143000	0.959
5	MV112	Final Mission	Action	8	190000	0.689
6	MV101	The Last Horizon	Science Fiction	8	125000	0.680
7	MV113	Parallel Lives	Drama	8	118000	0.645
8	MV103	Hidden Truth	Thriller	8	110000	0.643
9	MV109	Midnight Journey	Adventure	8	132000	0.634
10	MV110	Laugh Again	Comedy	8	105000	0.627
11	MV102	City Lights	Comedy	8	98000	0.624
12	MV106	Silent River	Drama	8	87000	0.597
13	MV114	Wild Trails	Adventure	7	99000	0.308

Figure 1.5 Sorted movie list showing **Popularity Rank**.

Movies were ranked successfully based on their popularity score, allowing easy identification of top-performing movies.

Conclusions

The Movie Rating Data Preprocessing project was successfully implemented using the Pandas library. The dataset was created with missing values, and the missing Rating values were replaced using the average rating. Records with missing MovieID or Genre were removed to improve data quality. The Rating column was converted into integer type, and the values were normalized using the Min-Max normalization technique. The project also identified top-rated movies, generated genre-wise reports, analysed viewer preferences, recommended

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

	movies based on genre and rating, and visualized movie popularity trends. This exercise provided practical knowledge of data preprocessing, data analysis, and dashboard development using Python, Pandas, and Streamlit.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018 2. Python Program (https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%202/EXERCISE_PROGRAM.py) 3. Class lecture link (https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view)

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 23/07/2026

Week No.: 3

Program S. No. 5

Title	Creating and Manipulating Arrays Using NumPy				
Problem Statement	Create a 2D array of shape (3, 4) with random integers between 0 and 10. Perform the following operations: calculate the mean and standard deviation of the entire array, slice the array to get the first two rows and last two columns, reshape the array to shape (4, 3), and perform element-wise multiplication with another array of the same shape				
Objectives	• To create and manipulate two-dimensional NumPy arrays. • To represent predefined array values using a Python dictionary. • To calculate the mean and standard deviation of the entire array. • To slice the array and extract selected rows and columns. • To reshape an array from shape (3, 4) to shape (4, 3). • To perform element-wise multiplication between arrays of the same shape. • To understand array shape, slicing, reshaping and numerical operations using NumPy.				
Dataset URL Link / File	Predefined dictionary dataset used directly inside the Python program:				
	Array	Row 1	Row 2	Row 3	Row 4
	Array1 (3 x 4)	2, 5, 7, 1	8, 3, 6, 4	9, 0, 5, 2	—
	Array2 (4 x 3)	1, 2, 3	4, 5, 6	7, 8, 9	2, 3, 4
	No external dataset file or URL is required.				
Functions Details	Sl. No.	Function	Package	Description of function	
	1	np.array()	NumPy	Converts dictionary values into NumPy arrays.	
	2	shape	NumPy	Returns the dimensions of an array as rows and columns.	
	3	np.mean()	NumPy	Calculates the mean of all elements in the array.	
	4	np.std()	NumPy	Calculates the standard deviation of all array elements.	
	5	Array slicing	NumPy	Extracts the first two rows and last two columns.	
	6	reshape()	NumPy	Changes the array shape from (3, 4) to (4, 3).	
	7	round()	Python	Rounds numerical output to the specified decimal places.	
	8	print()	Python	Displays arrays, calculations and headings in the console.	
	9	range()	Python	Creates a sequence of integer values when required.	
	10	len()	Python	Returns the number of items in an object.	
	11	dtype	NumPy	Shows the data type of array elements.	

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 23/07/2026

Week No.: 3

Program S. No. 5

	12	Element-wise *	NumPy	Multiplies corresponding elements of equal-shaped arrays.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import NumPy using import numpy as np. Step 2: Create a predefined dictionary containing Array1 and Array2. Step 3: Convert the dictionary values into NumPy arrays using np.array(). Step 4: Display Array1 and verify its shape as (3, 4). Step 5: Calculate the mean using np.mean(). Step 6: Calculate the standard deviation using np.std(). Step 7: Slice Array1 using array1[:2, -2:] to get the first two rows and last two columns. Step 8: Reshape Array1 to (4, 3) using reshape(4, 3). Step 9: Display Array2 and verify its shape as (4, 3). Step 10: Perform element-wise multiplication using reshaped_array * array2. Step 11: Display all results and their respective shapes. Step 12: Verify that all requested NumPy operations are completed successfully.			
Program	<pre>import numpy as np array_data = { "Array1": [[2, 5, 7, 1], [8, 3, 6, 4], [9, 0, 5, 2]], "Array2": [[1, 2, 3], [4, 5, 6], [7, 8, 9], [2, 3, 4]] } array1 = np.array(array_data["Array1"]) array2 = np.array(array_data["Array2"]) print("ORIGINAL 2D\nARRAY") print(array1) print("Shape:", array1.shape) mean_value = np.mean(array1) standard_deviation = np.std(array1) print("\nMean:", round(mean_value, 2)) print("Standard Deviation:", round(standard_deviation, 2)) sliced_array = array1[:2, -2:] print("\nFIRST TWO ROWS AND LAST TWO COLUMNS") print(sliced_array) print("Shape:", sliced_array.shape) reshaped_array = array1.reshape(4, 3) print("\nRESHAPED ARRAY")</pre>			

Student Reg. No: 26MML0031 — Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 23/07/2026

Week No.: 3

Program S. No. 5

```
print(reshaped_array)
print("Shape:", reshaped_array.shape)

print("\nSECOND ARRAY")
print(array2)

multiplied_array = reshaped_array * array2
print("\nELEMENT-WISE
MULTIPLICATION")
print(multiplied_array)
print("Shape:", multiplied_array.shape)

print("\nFINAL SUMMARY")
print("Original Shape:",
array1.shape) print("Mean:",
round(mean_value, 2))
print("Standard Deviation:", round(standard_deviation, 2))
print("Sliced Shape:", sliced_array.shape)
print("Reshaped Shape:", reshaped_array.shape)
print("Multiplied Shape:", multiplied_array.shape)
```

Screenshot 1: Original Array and Shape

ORIGINAL 2D ARRAY

```
=====
[[2 5 7 1]
 [8 3 6 4]
 [9 0 5 2]]
```

Shape of Array: (3, 4)

Figure 1.1 Original 2D Array

The predefined dictionary values were converted into a NumPy array. The output confirms that the array contains 3 rows and 4 columns.

Screenshot 2: Mean and Standard Deviation

MEAN OF THE ENTIRE ARRAY

```
=====
Mean: 4.33
```

STANDARD DEVIATION OF THE ENTIRE ARRAY

```
=====
Standard Deviation: 2.72
```

Figure 1.2 Mean & Standard Deviation of the Array

The mean represents the average of all elements in the array, while the standard deviation shows how widely the values are distributed around the mean.

**Output
Snapshots
With
Explanation/
Observations**

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 23/07/2026

Week No.: 3

Program S. No.

5 Screenshot 3: Array Slicing

```
FIRST TWO ROWS AND LAST TWO COLUMNS
```

```
=====
[[7 1]
 [6 4]]
```

Figure 1.3 Sliced Array

Array slicing was performed using `array1[:2, -2:]`. The resulting sliced array has a shape of 2×2 .

Screenshot 4: Reshaped Array

```
RESHAPED ARRAY
```

```
=====
[[2 5 7]
 [1 8 3]
 [6 4 9]
 [0 5 2]]
```

```
Shape After Reshaping: (4, 3)
```

Figure 1.4 Reshaped Array

The `reshape()` function changed the arrangement of the 12 elements without changing their values or total count.

Screenshot 5: Element-wise Multiplication

```
ELEMENT-WISE MULTIPLICATION
```

```
=====
[[ 2 10 21]
 [ 4 40 18]
 [42 32 81]
 [ 0 15  8]]
```

Figure 1.5 Element Wise Multiplication

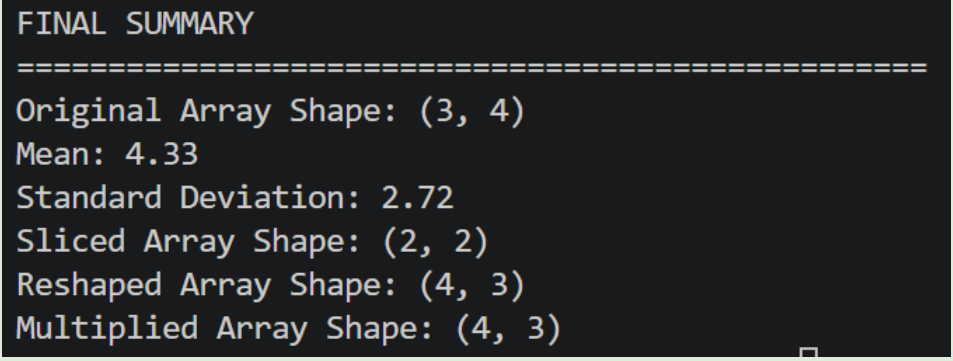
Corresponding elements of the two arrays with shape (4, 3) were multiplied using the `*` operator.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 23/07/2026

Week No.: 3

Program S. No. 5

	Screenshot 6: Final Summary  <i>Figure 1.6 Final Summary</i> The final output verifies that all statistical, slicing, reshaping and multiplication operations were completed successfully.
Conclusions	The NumPy array manipulation exercise was completed successfully. A two-dimensional array of shape (3, 4) was created using predefined dictionary data. The mean and standard deviation were calculated, the first two rows and last two columns were extracted through slicing, and the array was reshaped to (4, 3). Element-wise multiplication was performed with another array of the same shape. The exercise provided practical understanding of NumPy array creation, statistical calculations, slicing, reshaping and element-wise operations.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018. 2. NumPy documentation for array creation, statistics, slicing, reshaping and mathematical operations. 3. Python program link (https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%203/PRACTICE_PROGRAM.py) 4. Programming for Data Science (MACSE502) class lecture notes.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

Title	Warehouse Shipment and Logistics Analytics Using NumPy				
Problem Statement	A logistics company tracks shipments from 4 warehouses across 7 weeks. Create a 4 x 7 shipment matrix. Perform the following operations: (1) compute average shipments, (2) compute standard deviation, (3) slice the first two warehouses and last three weeks, (4) reshape the matrix into a 7 x 4 matrix, and (5) perform element-wise multiplication with a transportation-cost matrix of the same shape. As an extra practice mini project, develop a Supply Chain Analytics Dashboard that tracks inventory movement, optimizes transportation costs, identifies logistics bottlenecks, generates warehouse performance reports, and supports supply-chain planning.				
Objectives	• To create a 4 x 7 warehouse shipment matrix using NumPy. • To store shipment, transportation-cost, and capacity data locally using predefined Python dictionaries. • To calculate the mean and standard deviation of the complete shipment matrix. • To extract the first two warehouses and last three weeks using NumPy slicing. • To reshape the shipment matrix from 4 x 7 into 7 x 4 without changing its elements. • To calculate warehouse-week transportation expenditure using element-wise multiplication. • To track warehouse totals, weekly inventory movement, transportation cost, and capacity utilization. • To identify high-performing warehouses, cost-efficient warehouses, peak weeks, and logistics bottlenecks. • To develop an additional Streamlit dashboard for warehouse reporting and supply-chain planning. • Learning Outcome: Students understand numerical analytics in logistics management.				
Dataset URL Link / File	The dataset is stored locally inside the console and dashboard programs using predefined dictionaries. No external CSV file, URL, or database is required. A. Shipment Matrix (units shipped)				
		Warehouse 1	120	135	128
		Warehouse 2	98	105	110
		Warehouse 3	140	145	150
		Warehouse 4	80	85	90

Student Reg. No.: 26MML0031

Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

B. Transportation Cost Matrix (cost per shipment unit)

Warehouse 1	12	12	13	13	14	14	15
Warehouse 2	10	11	11	12	12	13	13
Warehouse 3	14	14	15	15	16	16	17
Warehouse 4	9	9	10	10	11	11	12

C. Weekly Capacity Dictionary

Warehouse	Weekly Capacity
Warehouse 1	175
Warehouse 2	145
Warehouse 3	180
Warehouse 4	120

Functions Details

Sl. No.	Function	Package	Description of function
1	np.array()	NumPy	Converts dictionary values into two-dimensional NumPy matrices.
2	np.mean()	NumPy	Calculates average shipments for the full matrix or a selected axis.
3	np.std()	NumPy	Measures the variation of shipment values around the mean.
4	Array slicing	NumPy	Extracts the first two warehouses and last three weeks.
5	reshape()	NumPy	Reorganizes the 4 x 7 matrix into a 7 x 4 matrix.
6	np.multiply()	NumPy	Performs element-wise multiplication of shipments and unit costs.
7	np.sum()	NumPy	Calculates warehouse totals, weekly totals, and total transportation cost.
8	np.argmax()	NumPy	Returns the index of the maximum shipment or utilization value.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

	9	np.where()	NumPy	Identifies warehouse-week combinations that cross the bottleneck threshold.
	10	pd.DataFrame()	Pandas	Creates labeled warehouse and weekly reports for dashboard display.
	11	st.metric()	Streamlit	Displays important supply-chain key performance indicators.
	12	st.line_chart()/bar_chart()	Streamlit	Visualizes inventory movement, cost, utilization, and warehouse performance.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import NumPy for numerical operations. Step 2: Create local dictionaries containing warehouse names, week names, shipment data, transportation costs, and weekly capacities. Step 3: Convert dictionary values into NumPy arrays. Step 4: Display the original 4 x 7 shipment and transportation-cost matrices. Step 5: Calculate the mean of the entire shipment matrix using np.mean(). Step 6: Calculate the standard deviation using np.std(). Step 7: Slice shipments[:, :2, -3:] to obtain the first two warehouses and last three weeks. Step 8: Reshape the 4 x 7 shipment matrix into a 7 x 4 matrix. Step 9: Use np.multiply() to multiply each shipment value by its corresponding transportation cost. Step 10: Calculate warehouse-wise shipments, week-wise shipments, total costs, average costs, and capacity utilization. Step 11: Identify the highest-performing warehouse and peak shipment week using np.argmax(). Step 12: Identify logistics bottlenecks where utilization is at least 90 percent of weekly capacity. Step 13: Generate a warehouse performance report for console output. Step 14: Create a Streamlit dashboard for inventory movement, transportation-cost analysis, bottleneck reporting, and planning. Step 15: Execute the console program and dashboard, then capture each required output screenshot.			
Program	A. Console Program <pre>import numpy as np # Local dataset stored using predefined dictionaries shipment_data = { "Warehouse 1": [120, 135, 128, 142, 150, 160, 155], "Warehouse 2": [98, 105, 110, 115, 120, 118, 125], "Warehouse 3": [140, 145, 150, 155, 165, 170, 175], "Warehouse 4": [80, 85, 90, 95, 100, 105, 110] } transport_cost_data = { "Warehouse 1": [12, 12, 13, 13, 14, 14, 15], "Warehouse 2": [10, 11, 11, 12, 12, 13, 13],</pre>			

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

```
"Warehouse 3": [14, 14, 15, 15, 16, 16, 17],
"Warehouse 4": [9, 9, 10, 10, 11, 11, 12]
}

capacity_data =
{ "Warehouse 1":
  175,
  "Warehouse 2": 145,
  "Warehouse 3": 180,
  "Warehouse 4": 120
}

warehouses = np.array(list(shipment_data.keys()))
weeks = np.array(["Week 1", "Week 2", "Week 3", "Week 4",
                  "Week 5", "Week 6", "Week 7"])
shipments = np.array(list(shipment_data.values()))
transport_cost =
np.array(list(transport_cost_data.values())) capacity =
np.array(list(capacity_data.values()))

print("WAREHOUSE SHIPMENT MATRIX")
print(shipments)
print("Shape:", shipments.shape)

print("\nTRANSPORTATION COST MATRIX")
print(transport_cost)
print("Shape:", transport_cost.shape)

average_shipments = np.mean(shipments)
shipment_std = np.std(shipments)

print("\nAVERAGE AND STANDARD DEVIATION")
print("Average shipments:", round(average_shipments, 2))
print("Standard deviation:", round(shipment_std, 2))

sliced_shipments = shipments[:2, -3:]
print("\nFIRST TWO WAREHOUSES AND LAST THREE WEEKS")
print(sliced_shipments)
print("Sliced shape:", sliced_shipments.shape)

reshaped_shipments = shipments.reshape(7,
4) print("\nRESHAPED SHIPMENT
MATRIX")
print(reshaped_shipments)
print("Reshaped shape:", reshaped_shipments.shape)
transportation_expense = np.multiply(shipments, transport_cost)
print("\nELEMENT-WISE TRANSPORTATION EXPENSE")
print(transportation_expense)
warehouse_totals = np.sum(shipments, axis=1)
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

```
weekly_totals = np.sum(shipments, axis=0)
warehouse_costs = np.sum(transportation_expense, axis=1)
average_unit_cost = warehouse_costs / warehouse_totals
utilization = shipments / capacity[:, None] * 100
maximum_utilization = np.max(utilization, axis=1)

best_warehouse_index = np.argmax(warehouse_totals)
peak_week_index = np.argmax(weekly_totals)
efficient_warehouse_index = np.argmin(average_unit_cost)
bottleneck_index = np.argmax(maximum_utilization)
bottleneck_points = np.where(utilization >= 90)

print("\nWAREHOUSE PERFORMANCE REPORT")
for i in range(len(warehouses)):
    print(warehouses[i])
    print(" Total shipments:", warehouse_totals[i])
    print(" Average weekly shipments:", round(np.mean(shipments[i]), 2))
    print(" Transportation expense:", warehouse_costs[i])
    print(" Average cost per shipment:", round(average_unit_cost[i], 2))
    print(" Maximum capacity utilization:",
          round(maximum_utilization[i], 2), "%")

print("\nWEEKLY INVENTORY MOVEMENT")
for i in range(len(weeks)):
    print(weeks[i], ":", weekly_totals[i])

print("\nLOGISTICS BOTTLENECK POINTS")
for warehouse_i, week_i in zip(*bottleneck_points):
    print(warehouses[warehouse_i], weeks[week_i],
          "Utilization:", round(utilization[warehouse_i, week_i], 2), "%")

print("\nSUPPLY CHAIN PLANNING SUMMARY")
print("Highest-performing warehouse:",
      warehouses[best_warehouse_index])
print("Peak shipment week:", weeks[peak_week_index])
print("Most cost-efficient warehouse:",
      warehouses[efficient_warehouse_index])
print("Highest bottleneck risk:", warehouses[bottleneck_index])
print("Total shipments:", np.sum(shipments))
print("Total transportation expense:", np.sum(transportation_expense))
```

B. Extra Practice: Streamlit Supply Chain Analytics Dashboard

```
import numpy as np
import pandas as pd
import streamlit as st
```

```
st.set_page_config(
    page_title="Supply Chain Analytics Dashboard",
    layout="wide"
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

```
)

st.title("Warehouse Shipment and Supply Chain Analytics")

# The dashboard uses the same local dictionary dataset
shipment_data = {
    "Warehouse 1": [120, 135, 128, 142, 150, 160, 155],
    "Warehouse 2": [98, 105, 110, 115, 120, 118, 125],
    "Warehouse 3": [140, 145, 150, 155, 165, 170, 175],
    "Warehouse 4": [80, 85, 90, 95, 100, 105, 110]
}

transport_cost_data = {
    "Warehouse 1": [12, 12, 13, 13, 14, 14, 15],
    "Warehouse 2": [10, 11, 11, 12, 12, 13, 13],
    "Warehouse 3": [14, 14, 15, 15, 16, 16, 17],
    "Warehouse 4": [9, 9, 10, 10, 11, 11, 12]
}

capacity_data = {
    "Warehouse 1": 175,
    "Warehouse 2": 145,
    "Warehouse 3": 180,
    "Warehouse 4": 120
}

weeks = ["Week 1", "Week 2", "Week 3", "Week 4",
         "Week 5", "Week 6", "Week 7"]

shipment_df = pd.DataFrame.from_dict(
    shipment_data, orient="index", columns=weeks
)
cost_rate_df = pd.DataFrame.from_dict(
    transport_cost_data, orient="index", columns=weeks
)
expense_df = shipment_df * cost_rate_df
capacity_series = pd.Series(capacity_data)
utilization_df = shipment_df.div(capacity_series, axis=0) * 100

warehouse_totals = shipment_df.sum(axis=1)
weekly_totals = shipment_df.sum(axis=0)
warehouse_expense = expense_df.sum(axis=1)
average_cost = warehouse_expense / warehouse_totals
maximum_utilization = utilization_df.max(axis=1)

performance_report = pd.DataFrame({
    "Total Shipments": warehouse_totals,
    "Average Weekly Shipments": shipment_df.mean(axis=1).round(2),
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

```
"Transportation Expense": warehouse_expense,
"Average Cost per Shipment": average_cost.round(2),
"Maximum Utilization (%)": maximum_utilization.round(2)
})

best_warehouse = warehouse_totals.idxmax()
peak_week = weekly_totals.idxmax()
cost_efficient = average_cost.idxmin()
bottleneck = maximum_utilization.idxmax()

selected_warehouse = st.sidebar.selectbox(
    "Select warehouse", shipment_df.index
)
threshold = st.sidebar.slider(
    "Bottleneck threshold (%)", 70, 100, 90
)

c1, c2, c3, c4 = st.columns(4)
c1.metric("Total Shipments", f"{int(shipment_df.values.sum());}")
c2.metric("Top Warehouse", best_warehouse)
c3.metric("Peak Week", peak_week)
c4.metric("Bottleneck Risk", bottleneck)

movement_tab, cost_tab, bottleneck_tab, report_tab, planning_tab
= st.tabs([
    "Inventory Movement", "Transportation
    Costs", "Logistics Bottlenecks", "Warehouse
    Reports", "Supply Chain Planning"
])

with movement_tab:
    st.subheader("Weekly Inventory Movement")
    st.line_chart(shipment_df.T)
    st.bar_chart(weekly_totals)
    st.dataframe(shipment_df, use_container_width=True)

with cost_tab:
    st.subheader("Transportation Expense by Warehouse")
    st.bar_chart(warehouse_expense)
    st.write("Most cost-efficient warehouse:", cost_efficient)
    st.dataframe(expense_df, use_container_width=True)

with bottleneck_tab:
    st.subheader("Capacity
    Utilization")
    st.line_chart(utilization_df.T)
    bottleneck_table = utilization_df[utilization_df >= threshold]
    st.dataframe(bottleneck_table, use_container_width=True)
    if bottleneck_table.notna().any().any():
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

```
st.warning("High-utilization warehouse-week combinations
detected.")
else:
    st.success("No bottlenecks detected at the selected threshold.")

with report_tab:
    st.subheader("Warehouse Performance Report")
    st.dataframe(performance_report, use_container_width=True)
    st.bar_chart(performance_report[["Total Shipments"]])

with planning_tab:
    st.subheader("Supply Chain Planning Summary")
    st.write("Selected warehouse:", selected_warehouse)
    st.write("Highest-performing warehouse:",
best_warehouse) st.write("Peak shipment week:",
peak_week) st.write("Most cost-efficient warehouse:",
cost_efficient) st.write("Highest bottleneck risk:",
bottleneck)
st.write(
    "Planning action: Review vehicle allocation, dispatch frequency, "
    "capacity, and route cost for high-utilization warehouses."
)
```

Run using: python -m streamlit run dashboard.py

**Output
Snapshots
With
Explanation/
Observations**

Screenshot 1: Original Shipment and Transportation-Cost Matrices

```
WAREHOUSE SHIPMENT MATRIX
[[120 135 128 142 150 160 155]
 [ 98 105 110 115 120 118 125]
 [140 145 150 155 165 170 175]
 [ 80  85  90  95 100 105 110]]
Shape: (4, 7)
```

```
TRANSPORTATION COST MATRIX
[[12 12 13 13 14 14 15]
 [10 11 11 12 12 13 13]
 [14 14 15 15 16 16 17]
 [ 9  9 10 10 11 11 12]]
Shape: (4, 7)
```

Figure 1.1 Original Shipment & Transportation Cost Matrix

Explanation/Observation: The local dictionary data was converted into two NumPy matrices of shape (4, 7), representing four warehouses across seven weeks.

Screenshot 2: Average Shipment and Standard Deviation

```
AVERAGE AND STANDARD DEVIATION
Average shipments: 126.64
Standard deviation: 26.66
```

Figure 1.2 Average & Standard Deviation

Explanation/Observation: The mean represents the average shipment volume across all warehouse-week observations, while the standard deviation measures shipment variation.

Screenshot 3: Sliced Shipment Matrix

```
FIRST TWO WAREHOUSES AND LAST THREE WEEKS
[[150 160 155]
 [120 118 125]]
Sliced shape: (2, 3)
```

Figure 1.3 Sliced Shipment Matrix

Explanation/Observation: The expression `shipments[:2, -3:]` selected the first two warehouses and the last three weeks, producing a matrix of shape (2, 3).

Screenshot 4: Reshaped 7 x 4 Shipment Matrix

```
RESHAPED SHIPMENT MATRIX
[[120 135 128 142]
 [150 160 155 98]
 [105 110 115 120]
 [118 125 140 145]
 [150 155 165 170]
 [175 80 85 90]
 [95 100 105 110]]
Reshaped shape: (7, 4)
```

Figure 1.4 Reshaped Matrix

Explanation/Observation: The reshape operation reorganized all 28 shipment values into seven rows and four columns without adding or removing data.

Screenshot 5: Element-wise Transportation Expense Matrix

```
ELEMENT-WISE TRANSPORTATION EXPENSE
[[1440 1620 1664 1846 2100 2240 2325]
 [ 980 1155 1210 1380 1440 1534 1625]
 [1960 2030 2250 2325 2640 2720 2975]
 [ 720  765  900  950 1100 1155 1320]]
```

Figure 1.5 Element Wise Matrix

Explanation/Observation: Each shipment quantity was multiplied by its corresponding transportation cost to estimate warehouse-week transportation expenditure.

Screenshot 6: Warehouse Performance and Bottleneck Report

```
WAREHOUSE PERFORMANCE REPORT
Warehouse 1
  Total shipments: 990
  Average weekly shipments: 141.43
  Transportation expense: 13235
  Average cost per shipment: 13.37
  Maximum capacity utilization: 91.43 %
Warehouse 2
  Total shipments: 791
  Average weekly shipments: 113.0
  Transportation expense: 9324
  Average cost per shipment: 11.79
  Maximum capacity utilization: 86.21 %
Warehouse 3
  Total shipments: 1100
  Average weekly shipments: 157.14
  Transportation expense: 16900
  Average cost per shipment: 15.36
  Maximum capacity utilization: 97.22 %
Warehouse 4
  Total shipments: 665
  Average weekly shipments: 95.0
  Transportation expense: 6910
  Average cost per shipment: 10.39
  Maximum capacity utilization: 91.67 %
```

Figure 1.6 Warehouse Performance Report

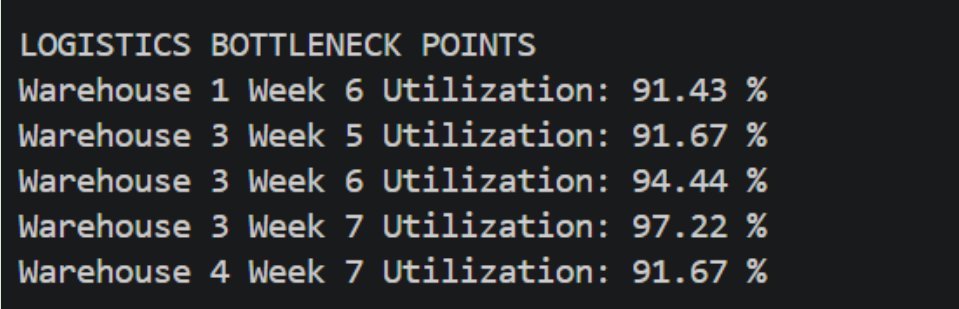
Explanation/Observation: The console report compared total shipments, average shipments, transportation expense, cost efficiency, and maximum capacity utilization for every warehouse.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

	Screenshot 7: Cost, Bottleneck, and Planning Views  <i>Figure 1.7 Logistics Bottleneck Points</i> Explanation/Observation: We visualized transportation expenses and capacity utilization, identified high-utilization periods, and generated planning recommendations.
Conclusions	The Warehouse Shipment and Logistics Analytics exercise was completed successfully using NumPy. Shipment, transportation-cost, and capacity data were stored locally using predefined dictionaries and converted into structured arrays. The mean and standard deviation of the shipment matrix were calculated, the first two warehouses and last three weeks were extracted through slicing, and the matrix was reshaped from 4 x 7 to 7 x 4. Element-wise multiplication was used to calculate transportation expenditure. Additional warehouse-wise and week-wise calculations identified the highest-performing warehouse, peak shipment week, most cost-efficient warehouse, and high-utilization bottlenecks.
References	1. NumPy documentation for array creation, descriptive statistics, slicing, reshaping, aggregation, and element-wise operations. 2. Pandas documentation for DataFrame creation and warehouse performance reporting. 3. Python Program Link https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%203/EXERCISE_PROGRAM.py

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

Title	Feature Engineering and Correlation Analysis Using Pandas			
Problem Statement	Create a DataFrame with columns Feature1, Feature2, and Target containing predefined sample data stored in a Python dictionary. Create a new feature that is the logarithm of Feature1, bin Feature2 into three categories (Low, Medium, and High), and calculate the correlation matrix of the DataFrame. Finally, create a scatter plot of Feature1 versus Feature2 distinguished by Target and interpret the visible patterns.			
Objectives	• To create a Pandas DataFrame using a predefined Python dictionary. • To understand feature engineering using logarithmic transformation. • To transform Feature1 using the natural logarithm function. • To divide Feature2 into Low, Medium, and High categories. • To calculate and interpret correlations among numerical variables. • To visualize the relationship between Feature1 and Feature2 using a scatter plot. • To distinguish Target classes using a black-and-white plotting format. • To interpret visible trends, class overlap, and feature relationships.			
Dataset URL Link / File	Predefined dictionary dataset used directly inside the Python program:			
	Feature1	Feature2	Target	
	5	10	0	
	8	18	0	
	12	15	1	
	15	25	0	
	18	30	1	
	22	28	1	
	25	35	0	
	30	40	1	
	35	38	1	
	40	45	0	
	45	50	1	
	50	55	1	
	55	60	0	
	60	65	1	
	70	75	1	
	No external CSV file, URL, database, or online dataset is required.			
Functions Details	Sl. No.	Function	Package	Description of function
	1	pd.DataFrame()	Pandas	Creates a DataFrame from the predefined dictionary data.
	2	np.log()	NumPy	Calculates the natural logarithm of each Feature1 value.
	3	pd.cut()	Pandas	Bins Feature2 into Low, Medium, and High categories.

Student Reg. No: 26MML0031 — Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

	4	corr()	Pandas	Calculates pairwise correlations between numerical columns.
	5	value_counts()	Pandas	Counts the number of records in each Feature2 category.
	6	sort_index()	Pandas	Displays category counts in their defined category order.
	7	round()	Python/ Pandas	Rounds logarithmic and correlation values for readable output.
	8	plt.figure()	Matplotlib	Creates a plotting area with the specified dimensions.
	9	plt.scatter()	Matplotlib	Creates the Feature1 versus Feature2 scatter plot.
	10	plt.xlabel()/plt.ylabel()	Matplotlib	Adds descriptive labels to the plot axes.
	11	plt.legend()	Matplotlib	Identifies the symbols used for each Target class.
	12	plt.show()	Matplotlib	Displays the completed scatter plot on the screen.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import Pandas, NumPy, and Matplotlib. Step 2: Define Feature1, Feature2, and Target values inside a Python dictionary. Step 3: Convert the dictionary into a DataFrame using pd.DataFrame(). Step 4: Display the original DataFrame and confirm the available columns. Step 5: Create Log_Feature1 by applying np.log() to Feature1. Step 6: Bin Feature2 into three equal-width categories using pd.cut(). Step 7: Label the categories as Low, Medium, and High. Step 8: Select the numerical columns required for correlation analysis. Step 9: Calculate and display the correlation matrix using corr(). Step 10: Count the number of observations in each Feature2 category. Step 11: Create a black-and-white scatter plot of Feature1 versus Feature2 grouped by Target. Step 12: Interpret the correlation, trend, and overlap visible in the output.			
Program	<pre>import pandas as pd import numpy as np import matplotlib.pyplot as plt # Predefined dictionary dataset data = { "Feature1": [5, 8, 12, 15, 18, 22, 25, 30, 35, 40, 45, 50, 55, 60, 70], "Feature2": [10, 18, 15, 25, 30, 28, 35, 40, 38, 45, 50, 55, 60, 65, 75], "Target": [0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1] }</pre>			

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

```
# Convert dictionary into a DataFrame
df = pd.DataFrame(data)
print("ORIGINAL DATAFRAME")
print(df)
print("DataFrame Shape:", df.shape)

# Create logarithmic feature
df["Log_Feature1"] = np.log(df["Feature1"])
print("\nDATAFRAME WITH LOG FEATURE")
print(df[["Feature1", "Log_Feature1"]].round(3))

# Bin Feature2 into three categories
df["Feature2_Category"] = pd.cut(
    df["Feature2"],
    bins=3,
    labels=["Low", "Medium", "High"]
)
print("\nFEATURE2 CATEGORIES")
print(df[["Feature2", "Feature2_Category"]])

# Calculate correlation matrix
numeric_columns = [
    "Feature1", "Feature2", "Target", "Log_Feature1"
]
correlation_matrix = df[numeric_columns].corr()
print("\nCORRELATION MATRIX")
print(correlation_matrix.round(3))

# Display category-wise
counts category_count = (
    df["Feature2_Category"]
    .value_counts()
    .sort_index()
)
print("\nFEATURE2 CATEGORY COUNT")
print(category_count)

# Create black-and-white scatter plot
class_zero = df["Target"] == 0
class_one = df["Target"] == 1

plt.figure(figsize=(8, 6))
plt.scatter(
    df.loc[class_zero, "Feature1"],
    df.loc[class_zero, "Feature2"],
    facecolors="white",
    edgecolors="black",
    marker="o",
```

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

```
label="Target = 0"
)
plt.scatter(
    df.loc[class_one, "Feature1"],
    df.loc[class_one, "Feature2"],
    color="black",
    marker="s",
    label="Target =
1"
)
plt.title("Feature1 vs. Feature2 Grouped by Target")
plt.xlabel("Feature1")
plt.ylabel("Feature2")
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend()
plt.tight_layout()
plt.show()

# Interpret visible patterns
feature_correlation = df["Feature1"].corr(df["Feature2"]) print("\
nPATTERN INTERPRETATION")
print(
    "Correlation between Feature1 and Feature2:",
    round(feature_correlation, 3)
)
print(
    "Feature1 and Feature2 have a strong positive "
    "relationship. Target classes overlap, although "
    "Target = 1 appears more often at medium and high values."
)
```

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

**Output
Snapshots
With
Explanation/
Observations**

Screenshot 1: Original DataFrame

ORIGINAL DATAFRAME

	Feature1	Feature2	Target
0	5	10	0
1	8	18	0
2	12	15	1
3	15	25	0
4	18	30	1
5	22	28	1
6	25	35	0
7	30	40	1
8	35	38	1
9	40	45	0
10	45	50	1
11	50	55	1
12	55	60	0
13	60	65	1
14	70	75	1

DataFrame Shape: (15, 3)

Figure 1.1 - Original DataFrame

Explanation/Observation: The dictionary was converted into a DataFrame containing 15 observations and the three required columns: Feature1, Feature2, and Target.

Screenshot 2: Logarithmic Feature

LOGARITHMIC FEATURE CREATED FROM FEATURE1

	Feature1	Log_Feature1
0	5	1.609
1	8	2.079
2	12	2.485
3	15	2.708
4	18	2.890
5	22	3.091
6	25	3.219
7	30	3.401
8	35	3.555
9	40	3.689
10	45	3.807
11	50	3.912
12	55	4.007
13	60	4.094
14	70	4.248

Figure 1.2 - Logarithmic Transformation of Feature1

Explanation/Observation: The natural logarithm of Feature1 was calculated using np.log(). The transformed values increase gradually and reduce the scale difference among larger values.

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

Screenshot 3: Feature2 Binning

FEATURE2 BINNED INTO LOW, MEDIUM AND HIGH

	Feature2	Feature2_Category
0	10	Low
1	18	Low
2	15	Low
3	25	Low
4	30	Low
5	28	Low
6	35	Medium
7	40	Medium
8	38	Medium
9	45	Medium
10	50	Medium
11	55	High
12	60	High
13	65	High
14	75	High

Figure 1.3 - Feature2 Categorized as Low, Medium, and High

Explanation/Observation: The `pd.cut()` function divided Feature2 into three equal-width categories. The output contains 6 Low, 5 Medium, and 4 High observations.

Screenshot 4: Correlation Matrix

CORRELATION MATRIX

	Feature1	Feature2	Target	Log_Feature1
Feature1	1.000	0.990	0.336	0.943
Feature2	0.990	1.000	0.314	0.946
Target	0.336	0.314	1.000	0.400
Log_Feature1	0.943	0.946	0.400	1.000

Figure 1.4 - Correlation Matrix of Numerical Features

Explanation/Observation: Feature1 and Feature2 have a very strong positive correlation of approximately 0.990. Log_Feature1 also has strong positive relationships with both original features.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No.

7 Screenshot 5: Category Count and Pattern Interpretation

CATEGORY COUNT AND PATTERN INTERPRETATION

Feature2_Category

Low 6

Medium 5

High 4

Correlation between Feature1 and Feature2: 0.990

Interpretation: Feature1 and Feature2 show a strong positive relationship. Target classes overlap, although Target = 1 occurs more often at medium and high feature values.

Figure 1.5 - Category Counts and Console Interpretation

Explanation/Observation: The category distribution and calculated correlation confirm an increasing relationship between Feature1 and Feature2. Target classes overlap, so the features do not completely separate the two classes.

Screenshot 6: Scatter Plot

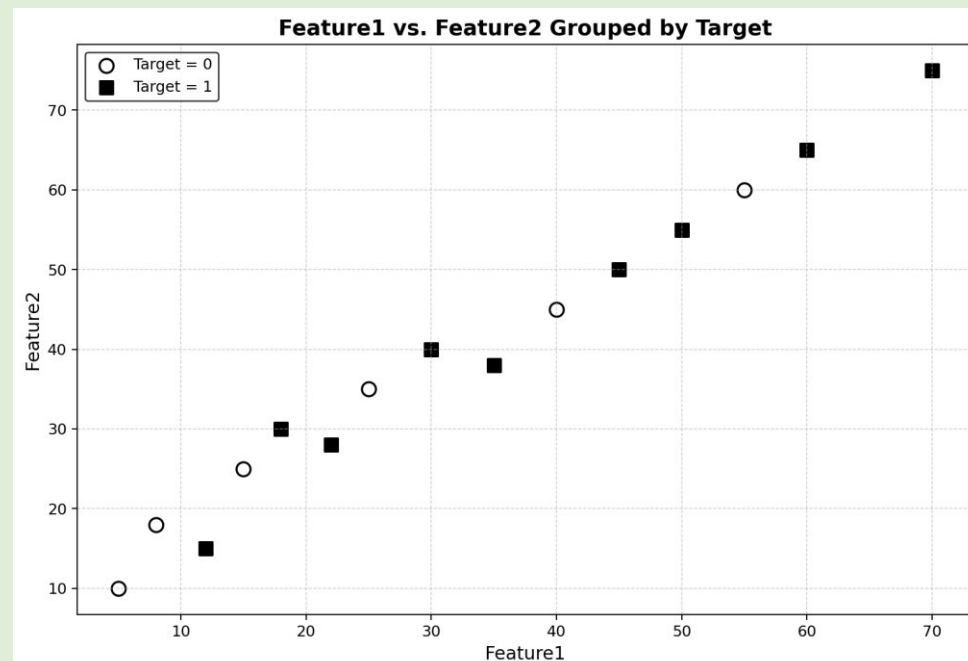


Figure 1.6 - Feature1 versus Feature2 Grouped by Target

Explanation/Observation: The points follow a clear upward pattern, showing that Feature2 generally increases as Feature1 increases. Black squares represent Target = 1, while white circles represent Target = 0. The classes overlap, but Target = 1 occurs more frequently at medium and high feature values.

Conclusions

The feature engineering and correlation analysis exercise was completed successfully using Pandas, NumPy, and Matplotlib. A DataFrame was created from a locally defined Python dictionary. A logarithmic feature was generated from Feature1, Feature2 was categorized into Low, Medium, and

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

	High groups, and a correlation matrix was calculated for the numerical variables. The correlation value of approximately 0.990 showed a very strong positive relationship between Feature1 and Feature2. The black-and-white scatter plot confirmed the upward trend and showed that the Target classes overlap, although Target = 1 appears more frequently at medium and high feature values. The exercise provided practical understanding of feature transformation, binning, correlation analysis, visualization, and pattern interpretation.
References	1. Pandas documentation for DataFrame creation, binning, category counting, and correlation analysis. 2. NumPy documentation for logarithmic transformations. 3. Matplotlib documentation for scatter plots, legends, labels, and plot formatting. 4. Python Program (https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%204/PRACTICE_PROGRAM.py) 5. Programming for Data Science (MACSE502) class lecture notes (https://drive.google.com/file/d/1PqsD1tC9aCgGFZo7QIJblBu1pIjg4UBs/view)

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

Title	Employee Attrition Prediction Using Workforce Indicators																																																																																																									
Problem Statement	A company wants to analyze workforce behavior and predict employee attrition. Create a DataFrame containing OvertimeHours, JobSatisfactionScore, SalaryLevel, TrainingHours, WorkLifeBalance, YearsAtCompany, PromotionCount, and AttritionStatus. Create Log_OvertimeHours, categorize JobSatisfactionScore as Disengaged, Satisfied, or Highly Engaged, generate a correlation matrix, create a treemap visualization of employee categories, and analyze workforce trends to identify practical retention insights.																																																																																																									
Objectives	• To create an employee workforce DataFrame from a predefined Python dictionary. • To perform logarithmic transformation on employee overtime hours. • To segment employees using job satisfaction scores. • To encode attrition status for numerical analysis. • To generate and interpret a correlation matrix of workforce indicators. • To analyze attrition rates across employee engagement categories. • To identify employees with high attrition risk using workforce conditions. • To create a treemap visualization of employee categories. • To derive retention insights related to workload, satisfaction, salary, training, promotion, and work-life balance.																																																																																																									
Dataset URL Link / File	Predefined dictionary dataset used directly inside the Python program: Table A: Overtime, Satisfaction, Salary and Training Indicators <table><tr><th>Emp.</th><th>Overtime Hours</th><th>Job Satisfaction Score</th><th>Salary Level</th><th>Training Hours</th></tr><tr><td>1</td><td>5</td><td>5</td><td>5</td><td>40</td></tr><tr><td>2</td><td>12</td><td>4</td><td>4</td><td>35</td></tr><tr><td>3</td><td>18</td><td>3</td><td>3</td><td>28</td></tr><tr><td>4</td><td>26</td><td>2</td><td>2</td><td>16</td></tr><tr><td>5</td><td>8</td><td>4</td><td>4</td><td>34</td></tr><tr><td>6</td><td>32</td><td>1</td><td>1</td><td>10</td></tr><tr><td>7</td><td>15</td><td>3</td><td>3</td><td>24</td></tr><tr><td>8</td><td>28</td><td>2</td><td>2</td><td>18</td></tr><tr><td>9</td><td>10</td><td>5</td><td>5</td><td>45</td></tr><tr><td>10</td><td>24</td><td>2</td><td>4</td><td>30</td></tr><tr><td>11</td><td>35</td><td>1</td><td>1</td><td>8</td></tr><tr><td>12</td><td>6</td><td>4</td><td>4</td><td>38</td></tr><tr><td>13</td><td>20</td><td>3</td><td>3</td><td>26</td></tr><tr><td>14</td><td>34</td><td>3</td><td>2</td><td>12</td></tr><tr><td>15</td><td>38</td><td>2</td><td>2</td><td>14</td></tr></table> Table B: Work-Life, Tenure, Promotion and Attrition Indicators <table><tr><th>Emp.</th><th>Work-Life Balance</th><th>Years at Company</th><th>Promotion Count</th><th>Attrition Status</th></tr><tr><td>1</td><td>5</td><td>10</td><td>3</td><td>No</td></tr><tr><td>2</td><td>4</td><td>7</td><td>2</td><td>No</td></tr><tr><td>3</td><td>3</td><td>5</td><td>1</td><td>No</td></tr><tr><td>4</td><td>2</td><td>2</td><td>0</td><td>Yes</td></tr></table>	Emp.	Overtime Hours	Job Satisfaction Score	Salary Level	Training Hours	1	5	5	5	40	2	12	4	4	35	3	18	3	3	28	4	26	2	2	16	5	8	4	4	34	6	32	1	1	10	7	15	3	3	24	8	28	2	2	18	9	10	5	5	45	10	24	2	4	30	11	35	1	1	8	12	6	4	4	38	13	20	3	3	26	14	34	3	2	12	15	38	2	2	14	Emp.	Work-Life Balance	Years at Company	Promotion Count	Attrition Status	1	5	10	3	No	2	4	7	2	No	3	3	5	1	No	4	2	2	0	Yes
Emp.	Overtime Hours	Job Satisfaction Score	Salary Level	Training Hours																																																																																																						
1	5	5	5	40																																																																																																						
2	12	4	4	35																																																																																																						
3	18	3	3	28																																																																																																						
4	26	2	2	16																																																																																																						
5	8	4	4	34																																																																																																						
6	32	1	1	10																																																																																																						
7	15	3	3	24																																																																																																						
8	28	2	2	18																																																																																																						
9	10	5	5	45																																																																																																						
10	24	2	4	30																																																																																																						
11	35	1	1	8																																																																																																						
12	6	4	4	38																																																																																																						
13	20	3	3	26																																																																																																						
14	34	3	2	12																																																																																																						
15	38	2	2	14																																																																																																						
Emp.	Work-Life Balance	Years at Company	Promotion Count	Attrition Status																																																																																																						
1	5	10	3	No																																																																																																						
2	4	7	2	No																																																																																																						
3	3	5	1	No																																																																																																						
4	2	2	0	Yes																																																																																																						

Student Reg. No: 26MML0031 Name: Makada Hasnain

— Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore —

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

	5	4	8	2	No	
	6	1	1	0	Yes	
	7	3	4	1	No	
	8	2	3	0	Yes	
	9	5	11	3	No	
	10	3	6	1	No	
	11	1	1	0	Yes	
	12	4	9	2	No	
	13	3	6	1	No	
	14	2	2	0	Yes	
	15	2	2	0	Yes	
	No external CSV file, URL, database, or online dataset is required.					
	Functions Details	Sl. No.	Function	Package	Description of function	
		1	pd.DataFrame()	Pandas	Creates the workforce DataFrame from the predefined dictionary.	
		2	np.log()	NumPy	Calculates the natural logarithm of OvertimeHours.	
		3	pd.cut()	Pandas	Categorizes job satisfaction and attrition risk into defined groups.	
4		map()	Pandas	Converts AttritionStatus values from Yes/No into 1/0.		
5		corr()	Pandas	Generates the correlation matrix for numerical workforce indicators.		
6		groupby()	Pandas	Groups employee records by engagement or attrition category.		
7		agg()	Pandas	Calculates employee count, attrition count, averages, and summaries.		
8		mean()	Pandas	Calculates overall attrition rate and average workforce indicators.		
9		astype()	Pandas	Converts Boolean risk conditions into integer risk points.		
10		value_counts()	Pandas	Counts employees within each attrition-risk category.		
11		Rectangle()	Matplotli b	Creates proportional rectangles for the treemap visualization.		
12		savefig()/show)	Matplotli b	Saves and displays the employee-category treemap.		
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import Pandas, NumPy, Matplotlib, and Rectangle from matplotlib.patches. Step 2: Create the workforce dataset locally in dictionary format. Step 3: Convert the dictionary into a DataFrame and assign employee numbers as the index. Step 4: Create Log_OvertimeHours using the natural logarithm of OvertimeHours. Step 5: Categorize JobSatisfactionScore as Disengaged, Satisfied, or Highly Engaged. Step 6: Encode AttritionStatus as AttritionFlag using No = 0 and Yes = 1.					

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

	Step 7: Select numerical workforce indicators and generate the correlation matrix. Step 8: Calculate the overall attrition rate and category-wise workforce summaries. Step 9: Compare average workforce indicators for employees who stayed and employees who left. Step 10: Create a rule-based risk score from overtime, satisfaction, salary, work-life balance, and promotion conditions. Step 11: Create a black-background treemap showing employee-category size and attrition rate. Step 12: Interpret workforce trends and recommend suitable employee-retention strategies.
Program	<pre>import pandas as pd import numpy as np import matplotlib.pyplot as plt from matplotlib.patches import Rectangle def print_heading(title): print("\n" + "=" * 100) print(title) print("=" * 100) # 1. Workforce dataset stored locally as a Python dictionary. employee_data = { "OvertimeHours": [5, 12, 18, 26, 8, 32, 15, 28, 10, 24, 35, 6, 20, 34, 38], "JobSatisfactionScore": [5, 4, 3, 2, 4, 1, 3, 2, 5, 2, 1, 4, 3, 3, 2], "SalaryLevel": [5, 4, 3, 2, 4, 1, 3, 2, 5, 4, 1, 4, 3, 2, 2], "TrainingHours": [40, 35, 28, 16, 34, 10, 24, 18, 45, 30, 8, 38, 26, 12, 14], "WorkLifeBalance": [5, 4, 3, 2, 4, 1, 3, 2, 5, 3, 1, 4, 3, 2, 2], "YearsAtCompany": [10, 7, 5, 2, 8, 1, 4, 3, 11, 6, 1, 9, 6, 2, 2], "PromotionCount": [3, 2, 1, 0, 2, 0, 1, 0, 3, 1, 0, 2, 1, 0, 0], "AttritionStatus": ["No", "No", "No", "Yes", "No", "Yes", "No", "Yes", "No", "No", "Yes", "No", "No", "Yes", "Yes"] } # 2. Create and display the DataFrame. df = pd.DataFrame(employee_data) df.index = range(1, len(df) + 1) df.index.name = "Employee" print_heading("ORIGINAL WORKFORCE DATAFRAME") print(df.to_string()) print("\nDataFrame Shape:", df.shape) # 3. Feature engineering and workforce segmentation. df["Log_OvertimeHours"] = np.log(df["OvertimeHours"]).round(3) df["EmployeeCategory"] = pd.cut(df["JobSatisfactionScore"], bins=[0, 2, 3, 5], labels=["Disengaged", "Satisfied", "Highly Engaged"], include_lowest=True)</pre>

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

```
df["AttritionFlag"] = df["AttritionStatus"].map({"No": 0, "Yes": 1})

print_heading("FEATURE ENGINEERING AND EMPLOYEE SEGMENTATION")
print(df[["OvertimeHours", "Log_OvertimeHours", "JobSatisfactionScore",
"EmployeeCategory", "AttritionStatus"]].to_string())

# 4. Correlation analysis.
numeric_columns = [
    "OvertimeHours", "JobSatisfactionScore", "SalaryLevel",
    "TrainingHours", "WorkLifeBalance", "YearsAtCompany",
    "PromotionCount", "Log_OvertimeHours", "AttritionFlag"
]
correlation_matrix = df[numeric_columns].corr()

print_heading("CORRELATION MATRIX")
print(correlation_matrix.round(3).to_string())

# 5. Workforce trend analysis.
overall_attrition_rate = df["AttritionFlag"].mean() * 100
category_summary = (
    df.groupby("EmployeeCategory", observed=False)
    .agg(
        EmployeeCount=("AttritionFlag", "size"),
        AttritionCount=("AttritionFlag", "sum"),
        AverageOvertime=("OvertimeHours", "mean"),
        AverageSalaryLevel=("SalaryLevel", "mean")
    )
    .reset_index()
)
category_summary["AttritionRatePercent"] = (
    category_summary["AttritionCount"]
    / category_summary["EmployeeCount"] * 100
).round(2)
category_summary["AverageOvertime"] =
category_summary["AverageOvertime"].round(2)
category_summary["AverageSalaryLevel"] =
category_summary["AverageSalaryLevel"].round(2)

status_summary = df.groupby("AttritionStatus")[["
    "OvertimeHours", "JobSatisfactionScore", "SalaryLevel",
    "TrainingHours", "WorkLifeBalance", "YearsAtCompany",
    "PromotionCount"
]].mean().round(2)

print_heading("WORKFORCE TREND
ANALYSIS") print("Overall Attrition
Rate:
{:.2f}%".format(overall_attrition_rate))
print("\nEmployee Category Summary")
print(category_summary.to_string(index=False))
print("\nAverage Workforce Indicators by Attrition Status")
print(status_summary.to_string())

# 6. Rule-based attrition risk analysis.
df["RiskScore"] = (
    (df["OvertimeHours"] >= 25).astype(int)
    + (df["JobSatisfactionScore"] <= 2).astype(int)
    + (df["SalaryLevel"] <= 2).astype(int)
    + (df["WorkLifeBalance"] <= 2).astype(int)
    + (df["PromotionCount"] == 0).astype(int)
)
df["AttritionRisk"] = pd.cut(
    df["RiskScore"], bins=[-1, 1, 4, 5], labels=["Low", "Medium",
"High"]
)
```

Student Reg. No: 26MML0031 — Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

```
high_risk_employees = df[df["AttritionRisk"] == "High"][["OvertimeHours", "JobSatisfactionScore", "SalaryLevel", "WorkLifeBalance", "PromotionCount", "RiskScore", "AttritionRisk", "AttritionStatus"]]

print_heading("ATTRITION RISK ANALYSIS")
print("Risk Category Distribution")
print(df["AttritionRisk"].value_counts().sort_index().to_string())
print("\nHigh-Risk Employees")
print(high_risk_employees.to_string())

# 7. Treemap visualization using proportional Matplotlib
rectangles, total_employees = category_summary["EmployeeCount"].sum()
fig, ax = plt.subplots(figsize=(11, 6))
fig.patch.set_facecolor("black")
ax.set_facecolor("black")
x_position = 0.0
grayscale_colors = ["#2F2F2F", "#555555", "#7A7A7A"]

for index, row in category_summary.iterrows():
    width = row["EmployeeCount"] / total_employees
    ax.add_patch(Rectangle(
        (x_position, 0), width, 1,
        facecolor=grayscale_colors[index], edgecolor="white",
        linewidth=2
    ))
    label = (
        f'{row["EmployeeCategory"]}\n'
        f'Employees: {int(row["EmployeeCount"])}\n'
        f'Attrition Rate: {row["AttritionRatePercent"]:.1f}%'
    )
    ax.text(
        x_position + width / 2, 0.5, label,
        color="white", ha="center",
        va="center", fontsize=12,
        fontweight="bold"
    )
    x_position += width

ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.axis("off")
ax.set_title(
    "Treemap of Employee Engagement Categories",
    color="white", fontsize=16, fontweight="bold", pad=18
)
plt.tight_layout()
plt.savefig(
    "hr_employee_category_treemap.png",
    dpi=300, facecolor="black", bbox_inches="tight"
)
plt.show()

# 8. Final interpretation.
attrition_correlations = (
    correlation_matrix["AttritionFlag"]
    .drop("AttritionFlag")
    .sort_values(ascending=False)
)
print_heading("FINAL WORKFORCE INSIGHTS")
print("Correlation of Workforce Indicators with Attrition")
print(attrition_correlations.round(3).to_string())
print("\nInterpretation:")
print(
    "1. Higher overtime is associated with greater attrition risk.\n"
```

Student Reg. No: 26MML0031 — Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

```
"2. Lower job satisfaction, salary level, work-life balance,
training,\n"
" tenure, and promotion opportunities are associated
with attrition.\n"
"3. Disengaged employees show the highest attrition rate.\n"
"4. Retention strategies should prioritize workload control,
employee\n"
" engagement, career growth, training, and work-life balance."
)
```

**Output
Snapshots
With
Explanation/
Observations**

Screenshot 1: Original Workforce DataFrame

Employee	OverTimeHours	JobSatisfactionScore	SalaryLevel	TrainingHours	WorkLifeBalance	TenureCompany	PromotionCount	AttritionStatus
1	5	1	1	10	4	10	1	No
2	12	4	4	15	4	7	2	No
3	18	3	3	10	3	3	1	Yes
4	26	2	2	10	2	2	0	Yes
5	18	2	2	10	2	2	0	No
6	32	1	1	10	1	1	0	Yes
7	15	3	3	10	3	4	1	Yes
8	20	2	2	10	2	4	1	Yes
9	20	5	5	10	5	11	1	No
10	24	2	2	10	2	4	1	No
11	35	1	1	8	1	1	0	Yes
12	6	4	4	10	4	9	1	No
13	20	3	3	20	3	6	1	Yes
14	30	2	2	12	2	8	0	Yes
15	38	2	2	14	2	2	0	Yes

Figure 1.1 - Original Workforce DataFrame

Explanation/Observation: The locally defined dictionary was converted into a DataFrame containing 15 employee records and all eight required workforce indicators.

Screenshot 2: Feature Engineering and Employee Segmentation

Employee	OverTimeHours	Log_OvertimeHours	JobSatisfactionScore	EmployeeCategory	AttritionStatus
1	5	1.609	1	Highly Engaged	No
2	12	2.485	4	Highly Engaged	No
3	18	2.890	3	Satisfied	No
4	26	3.258	2	Disengaged	Yes
5	18	2.890	2	Highly Engaged	No
6	32	3.466	1	Disengaged	Yes
7	15	2.708	3	Satisfied	No
8	20	3.302	2	Disengaged	Yes
9	20	2.996	5	Highly Engaged	No
10	24	3.178	2	Disengaged	No
11	35	3.555	1	Disengaged	Yes
12	6	1.792	4	Highly Engaged	No
13	20	2.996	3	Satisfied	No
14	30	3.526	2	Satisfied	Yes
15	38	3.638	2	Disengaged	Yes

Figure 1.2 - Logarithmic Overtime Feature and Employee Categories

Explanation/Observation: Log_OvertimeHours was generated using np.log(). JobSatisfactionScore was categorized as Disengaged, Satisfied, or Highly Engaged, and AttritionStatus was retained for comparison.

Screenshot 3: Correlation Matrix

	OverTimeHours	JobSatisfactionScore	SalaryLevel	TrainingHours	WorkLifeBalance	TenureCompany	PromotionCount	Log_OvertimeHours	AttritionFlag
OverTimeHours	1.000	-0.364	-0.327	-0.321	-0.154	0.306	0.303	0.364	-0.854
JobSatisfactionScore	-0.364	1.000	0.393	0.305	0.305	0.305	0.327	-0.374	-0.729
SalaryLevel	-0.327	0.393	1.000	0.307	0.305	0.305	0.305	-0.374	-0.851
TrainingHours	-0.321	0.305	0.307	1.000	0.305	0.305	0.305	-0.374	-0.871
WorkLifeBalance	-0.154	0.305	0.305	0.305	1.000	0.305	0.305	-0.374	-0.871
TenureCompany	0.306	0.305	0.305	0.305	0.305	1.000	0.305	-0.374	-0.871
PromotionCount	0.303	0.327	0.305	0.305	0.305	0.305	1.000	-0.374	-0.871
Log_OvertimeHours	0.364	-0.374	-0.374	-0.374	-0.374	-0.374	-0.374	1.000	0.772
AttritionFlag	-0.854	-0.729	-0.851	-0.871	-0.871	-0.871	-0.871	0.772	1.000

Figure 1.3 - Correlation Matrix of Workforce Indicators

Explanation/Observation: OvertimeHours has a strong positive relationship with attrition, while satisfaction, salary, training, work-life balance, tenure, and promotion indicators show negative relationships with attrition.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

Screenshot 4: Workforce Trend Analysis

WORKFORCE TREND ANALYSIS							
Overall Attrition Rate: 40.00%							
Employee Category Summary							
EmployeeCategory	EmployeeCount	AttritionCount	AverageOvertime	AverageSalaryLevel	AttritionRatePercent		
Disengaged	6	5	16.50	2.00	83.33		
Satisfied	4	1	21.75	2.75	25.00		
Highly Engaged	5	0	8.20	4.40	0.00		
Average Workforce Indicators by Attrition Status							
AttritionStatus	OvertimeHours	JobSatisfactionScore	SalaryLevel	TrainingHours	WorkLifeBalance	YearsAtCompany	PromotionCount
No	13.11	3.67	3.89	33.33	3.78	7.33	1.78
Yes	32.17	1.83	1.67	13.00	1.67	1.83	0.00

Figure 1.4 - Attrition Rate and Workforce Category Summary

Explanation/Observation: The overall attrition rate is 40%. Disengage employees have the highest attrition rate and the highest average overtime while highly engaged employees show no attrition in the sample.

Screenshot 5: Attrition Risk Analysis

Attrition Risk Analysis								
Risk Category Distribution								
AttritionRisk								
Low	9							
Medium	1							
High	5							
High-Risk Employees								
Employee	OvertimeHours	JobSatisfactionScore	SalaryLevel	WorkLifeBalance	PromotionCount	RiskScore	AttritionRisk	AttritionStatus
4	26	2	2	2	0	5	High	Yes
6	32	1	1	1	0	5	High	Yes
9	28	2	2	2	0	5	High	Yes
11	35	1	1	1	0	5	High	Yes
15	38	2	2	2	0	5	High	Yes

Figure 1.5 - Rule-Based Attrition Risk Distribution

Explanation/Observation: Employees were assigned risk points based o overtime, satisfaction, salary, work-life balance, and promotion conditions Five employees were identified as high-risk, and all five have an observe attrition status of Yes.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

Screenshot 6: Employee Category Treemap

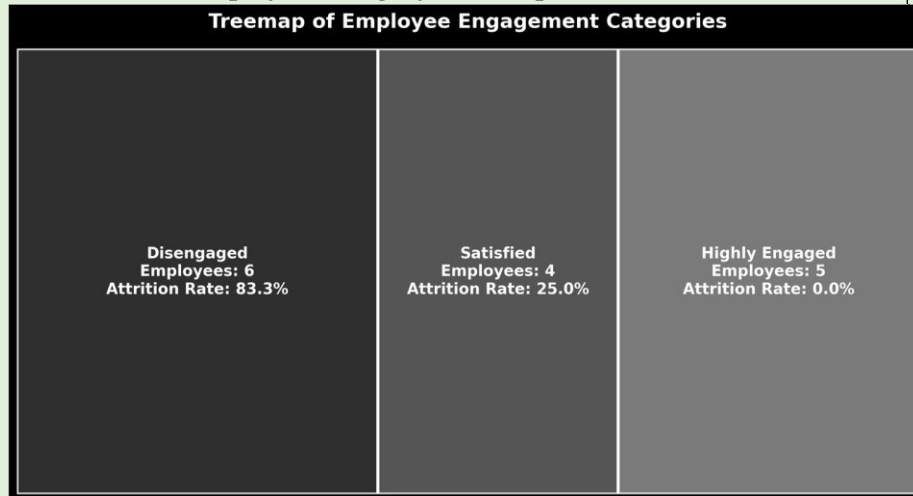


Figure 1.6 - Treemap of Employee Engagement Categories

Explanation/Observation: The treemap displays the relative size of each employee category. Each rectangle also reports the number of employees and the attrition rate for that category.

Screenshot 7: Final Workforce Insights

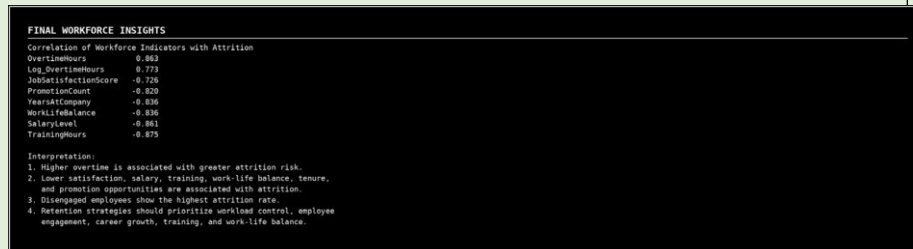


Figure 1.7 - Correlation-Based Workforce Interpretation

Explanation/Observation: The final analysis indicates that overtime is the strongest positive attrition indicator. Retention efforts should focus on workload control, engagement, compensation, training, career growth, and work-life balance.

Conclusions

The Human Resource Analytics exercise was completed successfully using Pandas, NumPy, and Matplotlib. A workforce DataFrame was created from a predefined Python dictionary, Log_OvertimeHours was generated, and employees were segmented using job satisfaction scores. The correlation matrix showed that higher overtime was positively associated with attrition, whereas job satisfaction, salary level, training, work-life balance, tenure, and promotion opportunities were negatively associated with attrition. The treemap highlighted the distribution of engagement categories, and the trend analysis showed that disengaged employees had the highest attrition rate. The rule-based risk analysis identified employees requiring immediate attention. These findings support retention strategies focused on workload reduction, employee

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

	engagement, fair compensation, career progression, training opportunities, and improved work-life balance.
References	1. Pandas documentation for DataFrame creation, categorization, grouping, aggregation, mapping, and correlation analysis. 2. NumPy documentation for logarithmic transformation. 3. Matplotlib documentation for figures, rectangles, text, and image export. 4. Python program supplied with this exercise https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%204/EXERCISE_PROGRAM.py 5. Programming for Data Science (MACSE502) class lecture notes https://drive.google.com/file/d/1PqsD1tC9aCgGFZo7QIJb1Bu1pIjg4UBs/view